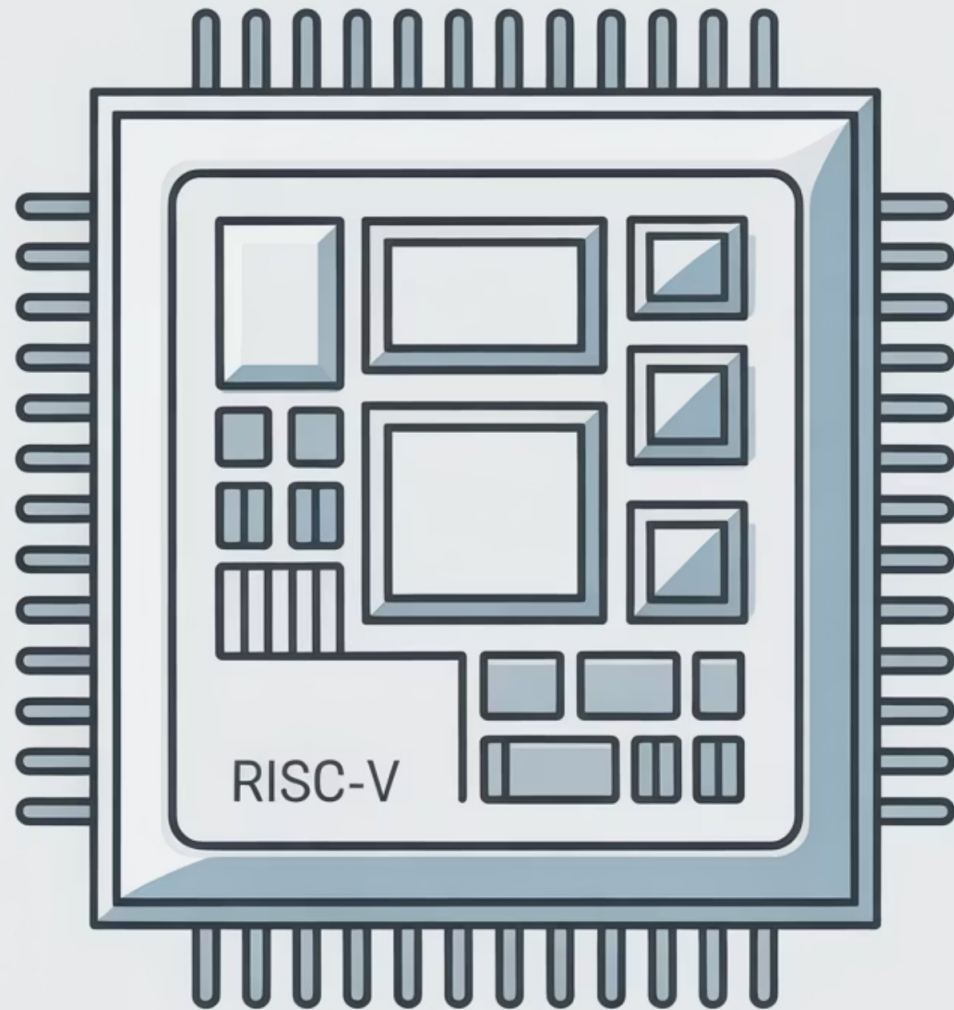




Functional Verification of RISC-V Designs

Dr. Oscar Palomar - BSC

Summer School on RISC-V Hardware–Software Co-Design.05/09/2026

Talk structure

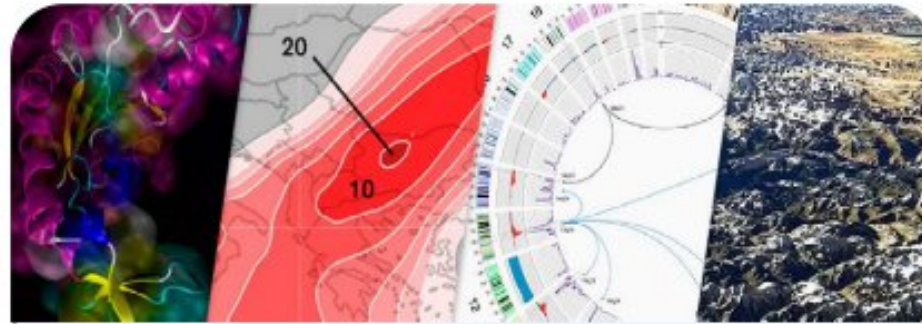
- Intro to RISC-V design and verification @ BSC
 - Key functional verification concepts
 - Implication of RISC-V features/characteristics for functional verification
 - Overview of a RISC-V verification flow
-
- Assuming (basic) knowledge of RISC-V and Computer Architecture (please stop me if needed)

Barcelona Supercomputing Center Centro Nacional de Supercomputación

BSC-CNS objectives



Supercomputing services to Spanish and EU researchers



R&D in Computer, Life, Earth and Engineering Sciences



PhD programme, technology transfer, public engagement

**BSC-CNS is
a consortium
that includes**

Spanish Government

60%



Catalan Government

30%



U. Polit. de Catalunya (UPC)

10%



MareNostrum 5

Total peak performance: **315.2 Pflops**

General Purpose Partition:	46.4 Pflops	(Apr 2024)
Accelerated Partition:	260 Pflops	(Apr2024)
Next Generation GPP:	2.82 Pflops	(Feb 2025)
Next Generation ACC:	6 Pflops	(TBA)



MareNostrum 1
2004 – 42.3 Tflops
1st Europe / 4th World
New technologies

MareNostrum 2
2006 – 94.2 Tflops
1st Europe / 5th World
New technologies

MareNostrum 3
2012 – 1.1 Pflops
12th Europe / 36th World

MareNostrum 4
2017 – 11.1 Pflops
2nd Europe / 13th World
New technologies

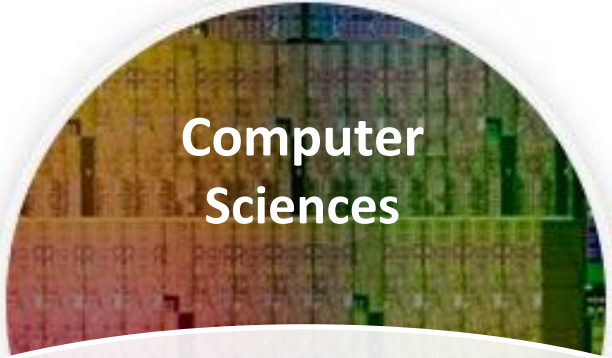
MareNostrum 5
2022
260 + 46.4 Pflops
8th and 19th World
3rd and 7th Europe



**Barcelona
Supercomputing
Center**
Centro Nacional de Supercomputación

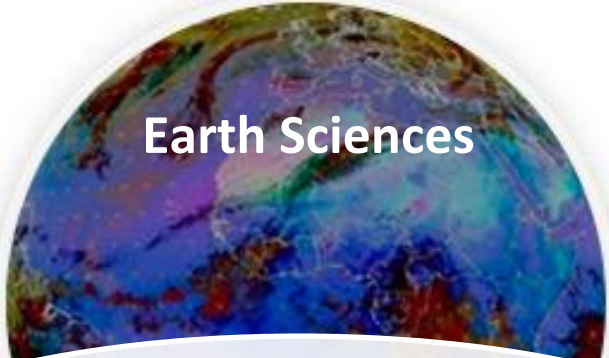


Mission of BSC Scientific Departments



Computer Sciences

To influence the way machines are built, programmed and used: programming models, performance tools, Big Data, Artificial Intelligence , computer architecture, energy efficiency



Earth Sciences

To develop and implement global and regional state-of-the-art models for short-term air quality forecast and long-term climate applications



Life Sciences

To understand living organisms by means of theoretical and computational methods (molecular modeling, genomics, proteomics)



CASE

To develop scientific and engineering software to efficiently exploit super-computing capabilities (biomedical, geophysics, atmospheric, energy, social and economic simulations)

CS mission and research directions

- **Mission:** influence the design of future of computational platforms targeting **HPC and AI** in scientific and embedded application domains, addressing both hardware and software aspects, as well as methodologies and evaluation
- **Five research directions:**



450 researchers, +25 research groups

RISC-V: An open standard ISA for/from the community

1

Why RISC-V?

Open standard, ISA royalty free

Foster **innovation**

Flexible and customizable standard based on extensions, profiles and platforms

Community driven

Rapidly building a **strong ecosystem**

2

What for?

Global collaboration opportunities

Many **funding opportunities** in Europe

Possibility of **influencing the EU tech strategy** in semiconductors & microelectronic

Unlock innovation by exploring new solutions for solving Scientific and societal challenges

3

BSC impact

Leading position in RISC-V activities in Europe through quantity and quality results based on projects

Expertise and knowledge in SW and HW stacks

Supporting HPC & AI convergence by providing specialized innovative solutions

Foster **talent attraction**

(Re-) **activating industrial ecosystem** around BSC know-how

Strengthening the ecosystem offering open base building blocks for anyone to build their own solutions

Country

- Tech sovereignty
- Accelerate local innovation and talent
- Incubate technology ecosystem from research to industry
- Access worldwide market

Multinational

- Control strategic roadmap
- New opportunities for innovation and influence
- Growth business models
- Avoid vendor lock-in

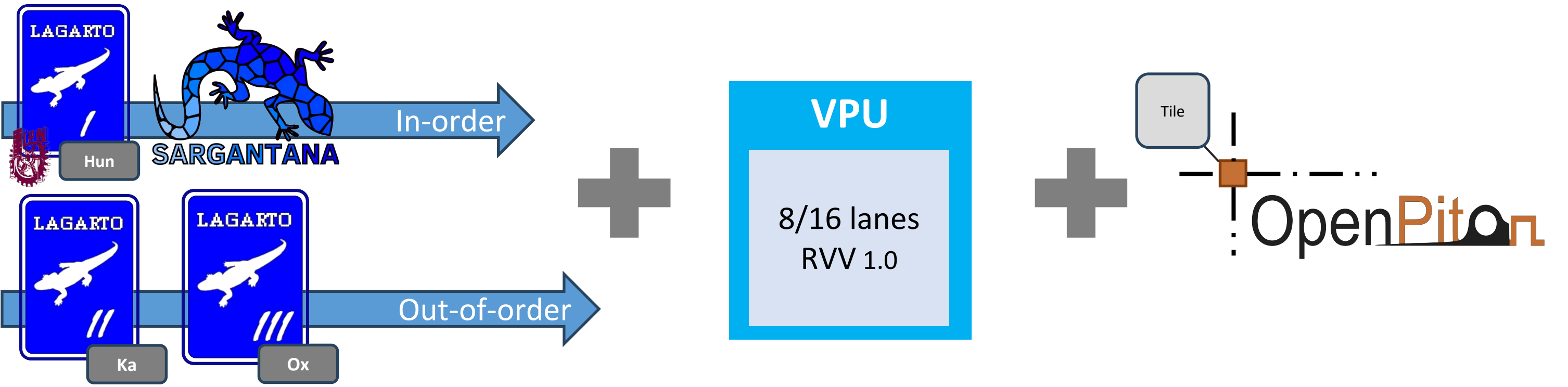
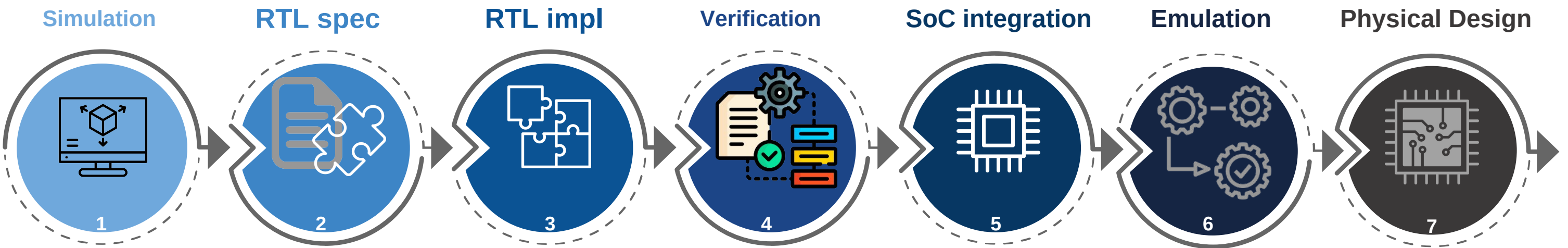
Research

- Collaborative ISA thought leadership
- Enables innovative research
- Access global RISC-V research network
- Tech transfer addressing real world applications

Start-ups

- Supercharge hardware and software co-design
- Accelerate strategic roadmap in greenfield applications
- Collaboration partners

RISC-V@BSC: Chip Design



Sargantana and more in:
<https://github.com/bsc-loca>

Lagarto RISC-V Tapeouts

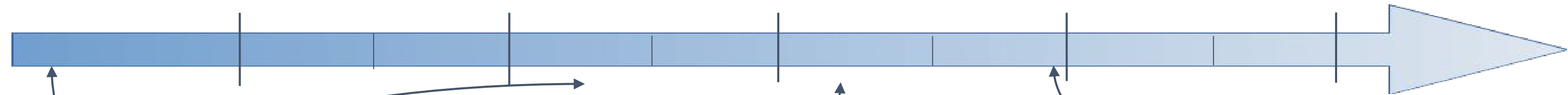
2019

2020

2021

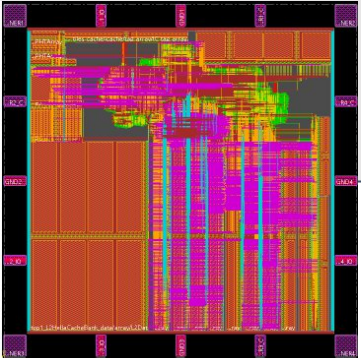
2022

2023



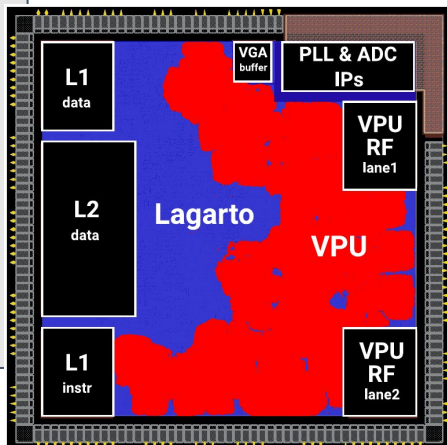
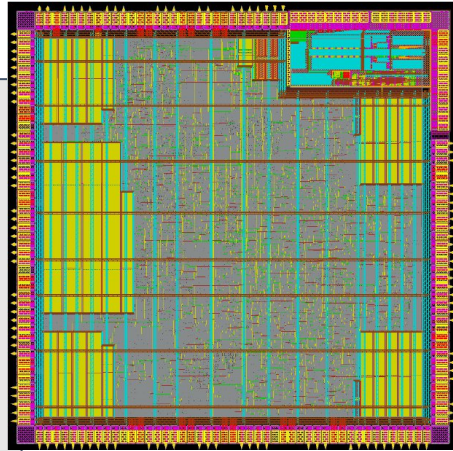
Lagarto (May'19)

- Lagarto Hun 5-stage in-order
- 150MHz (external)
- TSMC 65nm
- 2.5mm²



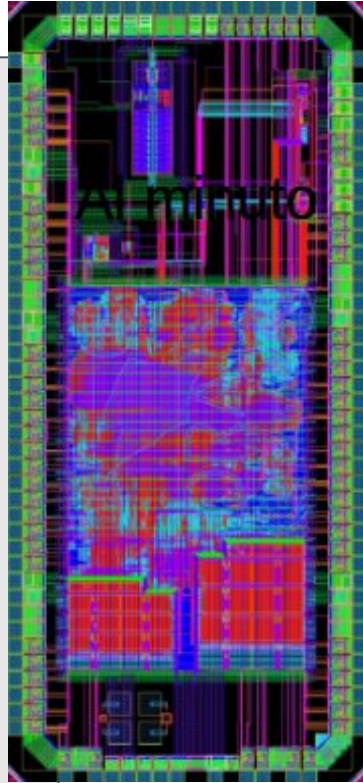
DVINO (Apr'21)

- Lagarto Hun in-order
- VPU
- PLL 600 MHz
- SDRAM mem cont
- HyperRAM
- VGA
- ADC
- TSMC 65nm
- 8mm²



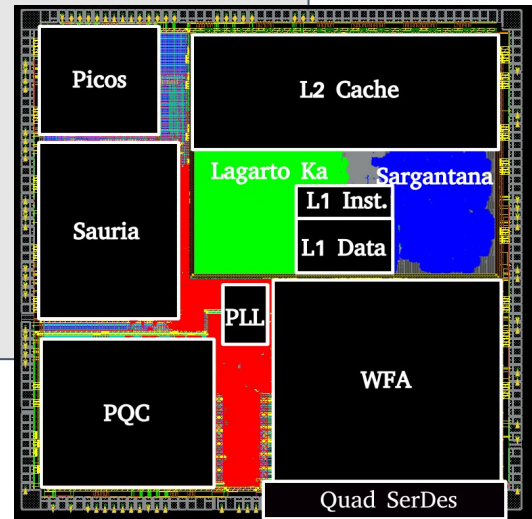
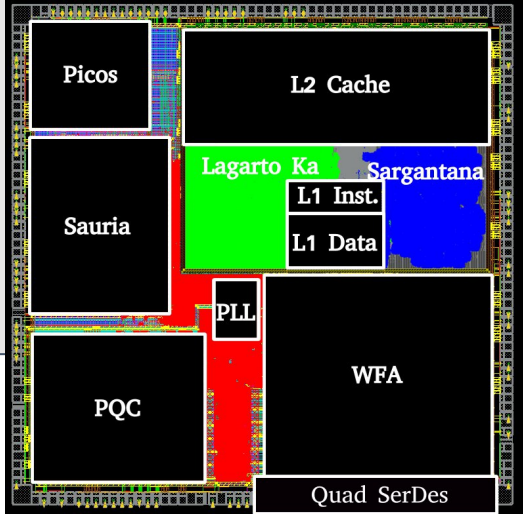
Sargantana (Feb'22)

- Sargantana 7-stage in-order
- PLL 1.2 GHz
- Custom extensions
- SDRAM
- Prototype analog IPs: SerDes 8GHz
- GF 22nm
- Area: 2.9mm²



Kameleon (Dec'22)

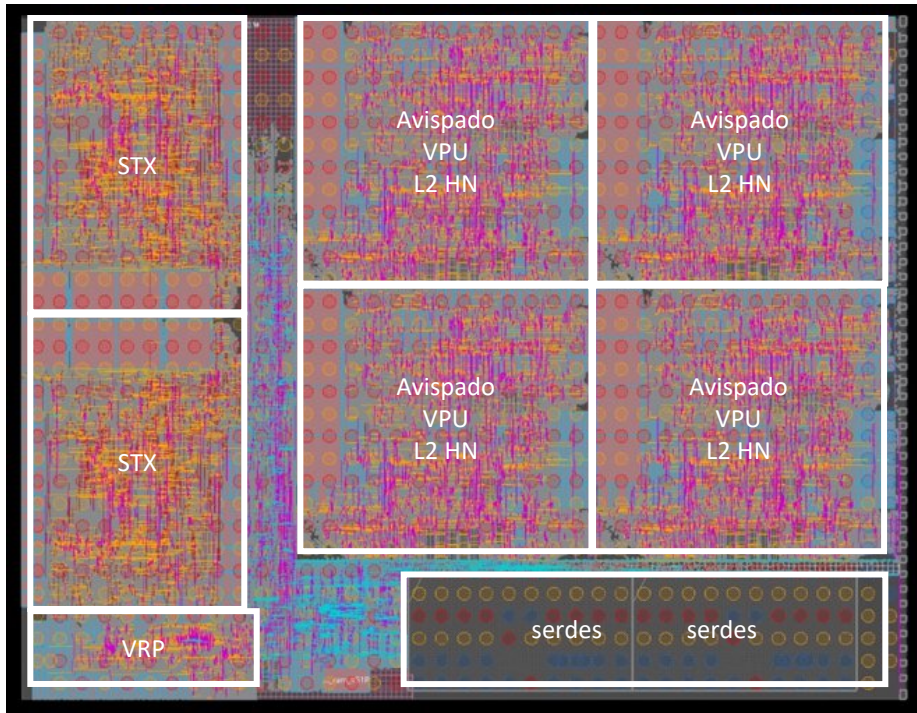
- Lagarto Ka 11-stage ooo
- PLL 1.2 GHz
- Automotive Accel.
- Crypto Accel.
- Genomics Accel.
- PICOS Accel.
- SerDes 8GHz
- GF 22nm
- Area: 9mm²



Contributing to EU chips



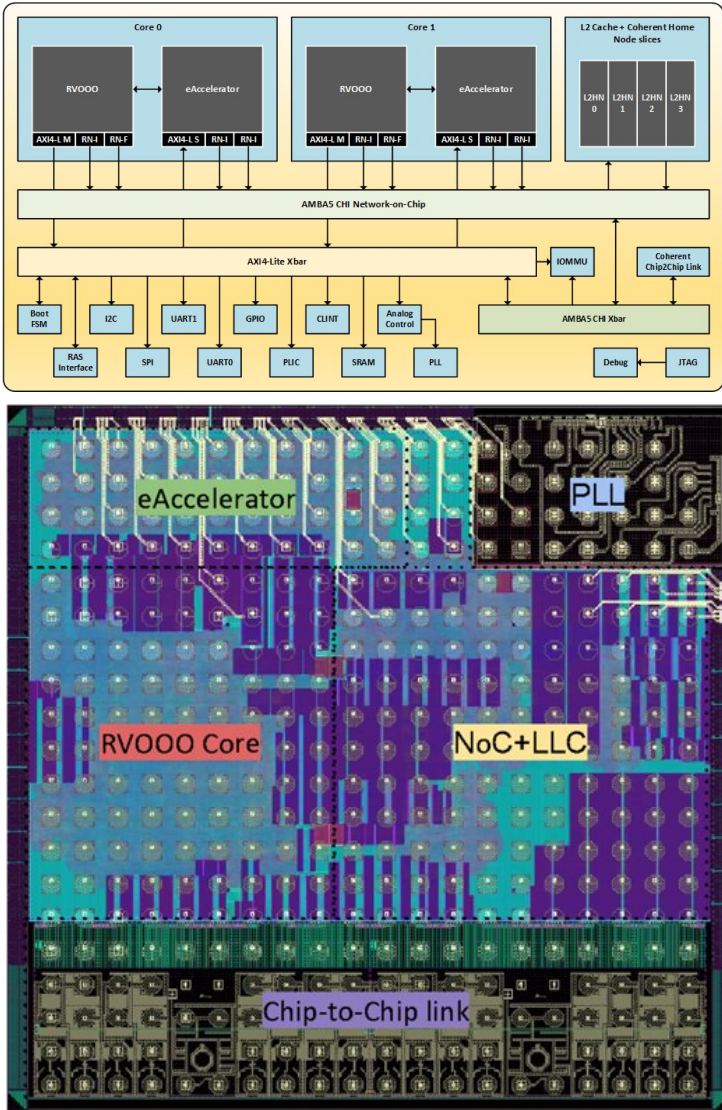
Vitruvius VPU in the EPI EPAC
(European Processor ACcelerator)



2021, 2022



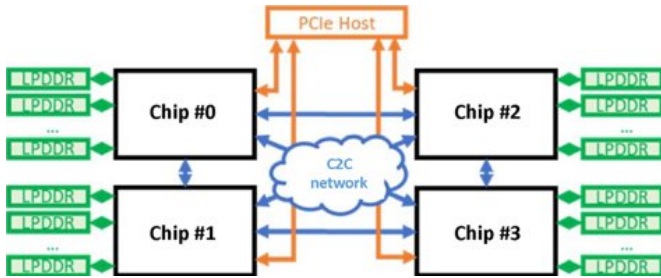
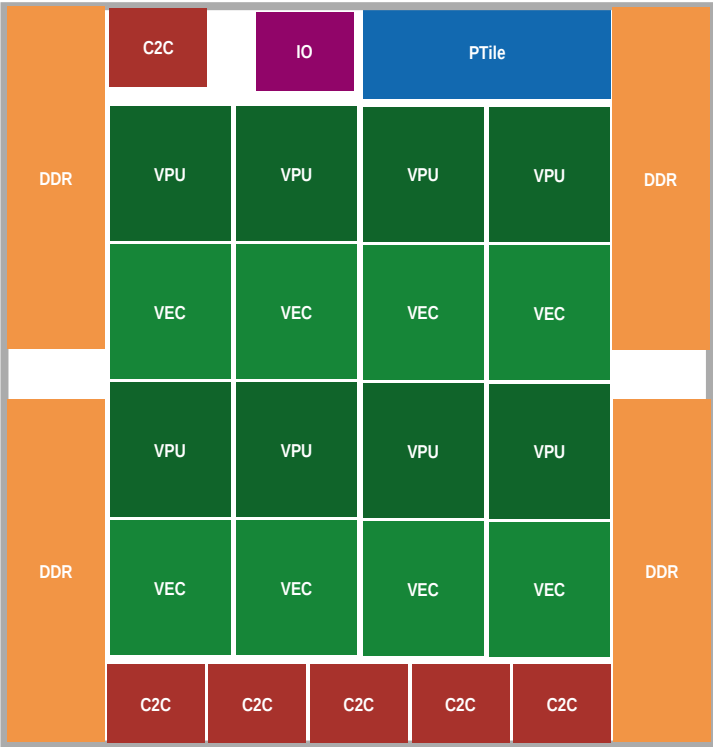
VPU in eProcessor eAccelerator



2024



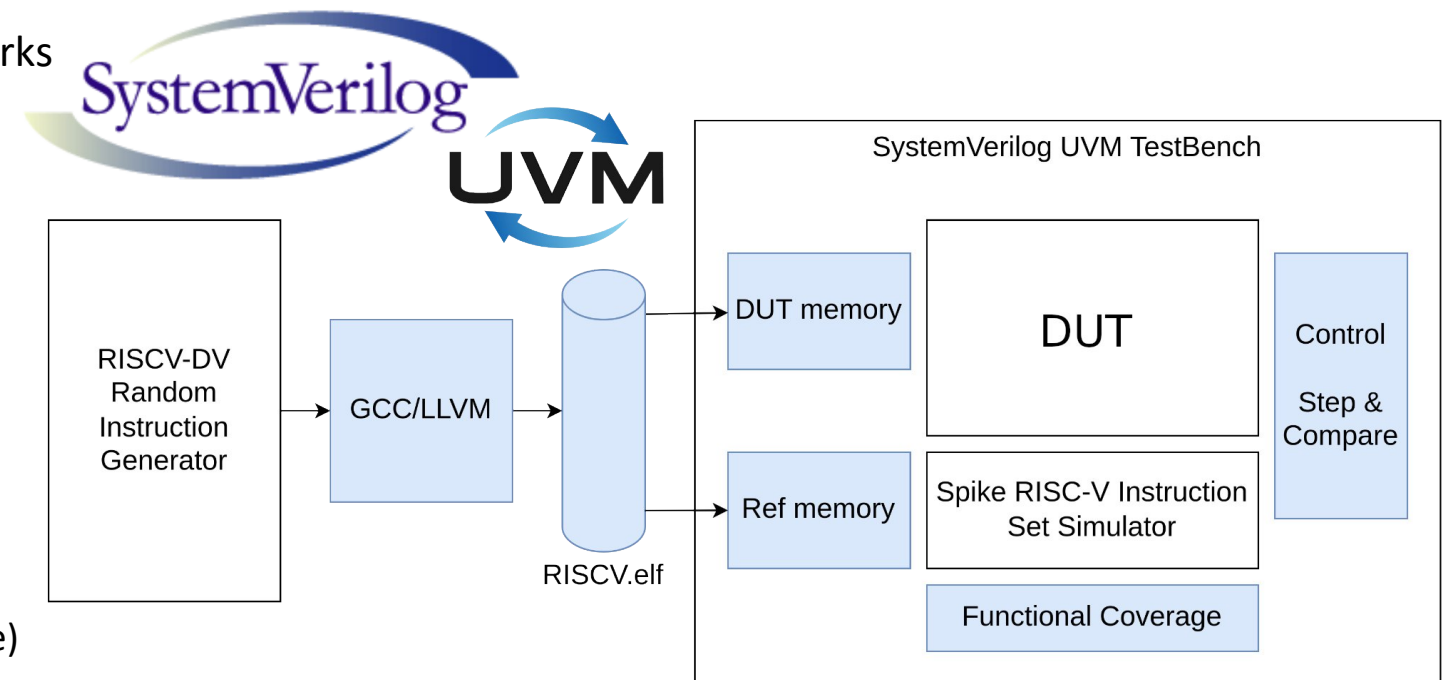
EUPilot VEC chip with 8 cores and
multichip support in GF12nm in 2025



2025

Validation

- Our mission: Verify RISC-V BSC designs
- Methodology: UVM environments with cosimulation (using open-source RISC-V ISA simulator Spike, heavily modified)
- Developed UVMs for: FPU, VPU (many variants), cores (Sargantana, Lagarto Ka and Lagarto Ox), instruction cache, cpu-subsystem (core/multi-core with cache hierarchy)
- Random tests with Riscv-dv binary generator (much enhanced)
- Also wrote/ported directed tests / microkernels / small benchmarks
 - e.g: RVV tests, 5Js, FCOV, ISA, riscv-vector-tests
- Some assertions and functional coverage
- Reuse as much as possible
 - Common vpu part (ISA) / per-project (interface)
 - Reuse vpu-uvm inside core-uvm inside cpu-subsystem-uvm
 - Unified repos and branches (spike, riscv-dv, core-uvm (sarg+ka))
 - Same tests everywhere (RVV tests, riscv-tests)
- Rely a lot on CI and the CI team support
 - MR regressions, trigger pipelines for related repos
 - Larger nightly regressions, even larger weekly regression (+coverage)
 - Packages, dashboard, make clone to use latest RTL...
- Working on many improvements!
 - Open-source core-uvm, cpu-subsystem-uvm + Verilator support
 - LLM-generated assertions + other AI-based improvements
 - Multi-core full scoreboard support
 - Improved coverage, assertions and tests
 - FPGA-based acceleration of UVM testbenches



Verification: What? Why? Who? When? How?

What: Does the implementation always work as intended?

ohttps://en.wikipedia.org/wiki/Pentium_FDIV_bug

Why: We want/need chips that work well!

Who: not the implementer (biased)

When: As soon as possible

How: many techniques. Important concepts:

UVM (reusable testbenches), cosimulation,

Constrained Random Testing, Coverage, Assertions, VIP,

Formal Verification

The correct value of the calculation is:

$$\frac{4,195,835}{3,145,727} = 1.333820449136241002$$

When converted to the hexadecimal value used by the processor, 4,195,835 = 0x4005FB and 3,145,727 = 0x2FFFFFF. The "5" in 0x4005FB triggers the access to the "empty" array cells. As a result, the value returned by a flawed Pentium processor is incorrect at or beyond four digits:^[8]

$$\frac{4,195,835}{3,145,727} = 1.333739068902037589$$

which is actually the value of $\frac{4,195,579}{3,145,727} = \frac{4,195,835 - 256}{3,145,727}$.

It is costly, but effective

Figure 1 - IC Verification and Validation Cost by Process Node Feature Size

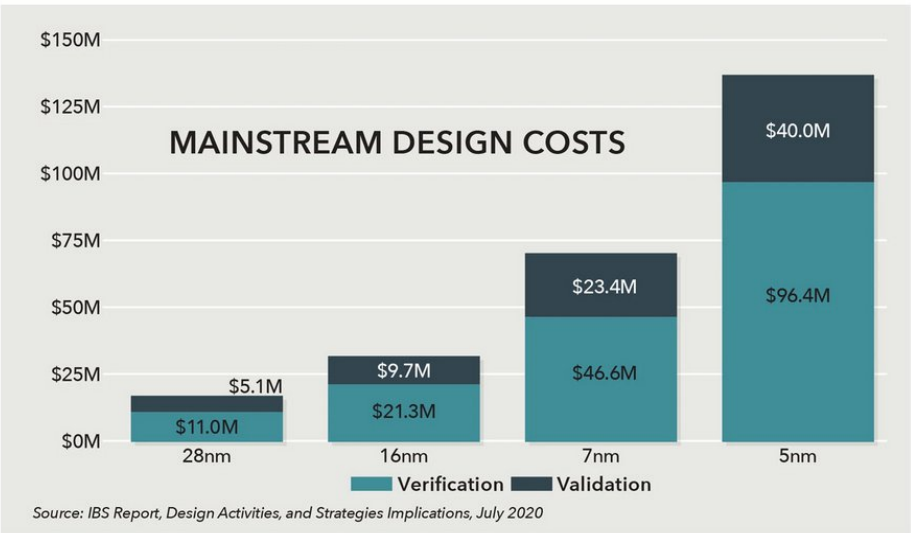
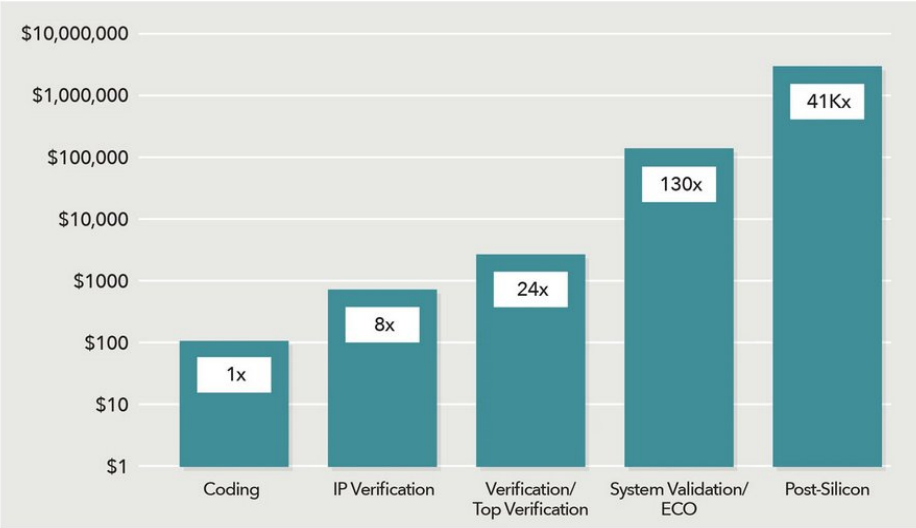
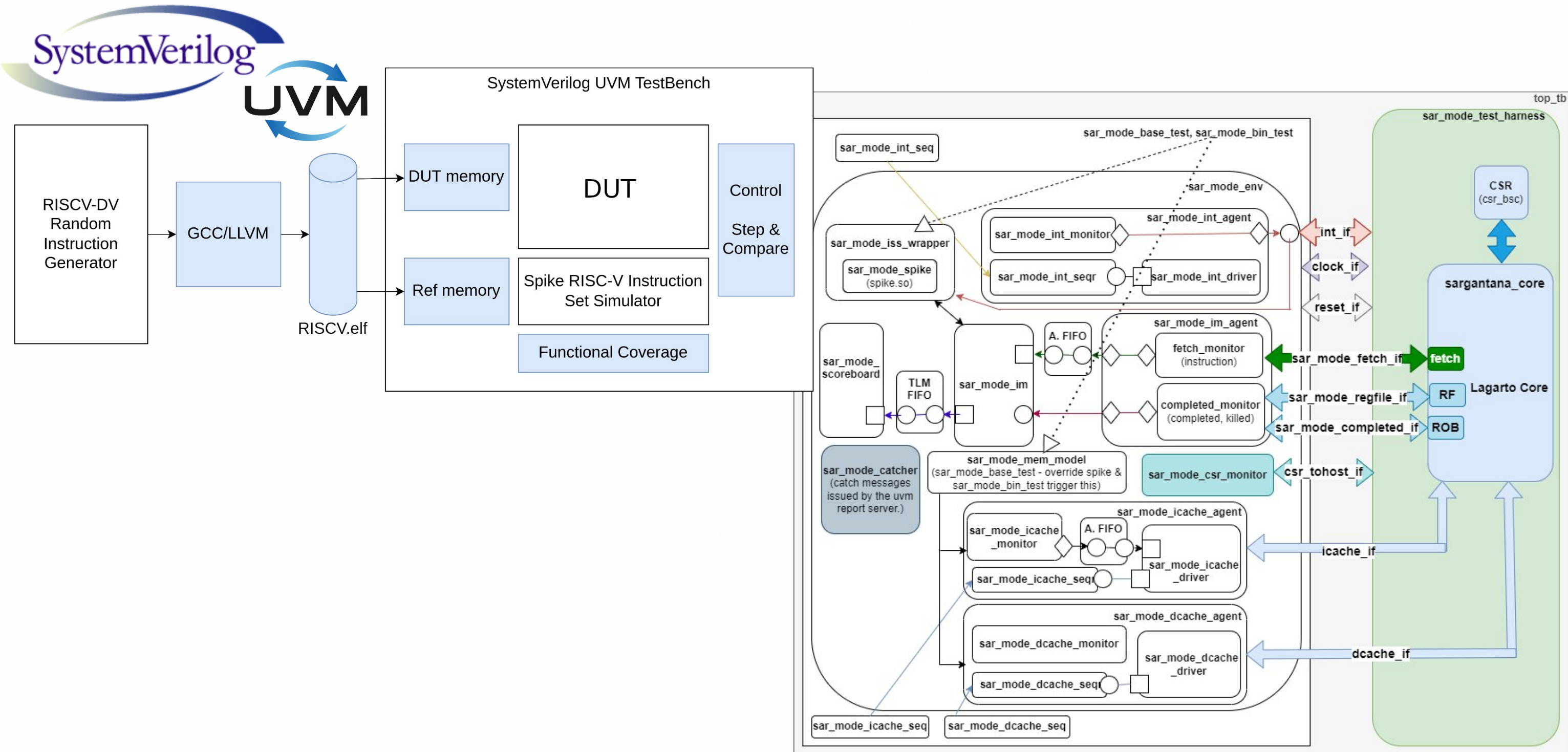


Figure 2 - Cost of Finding and Fixing a Bug at Various Development Stages for a 5 nm ASIC

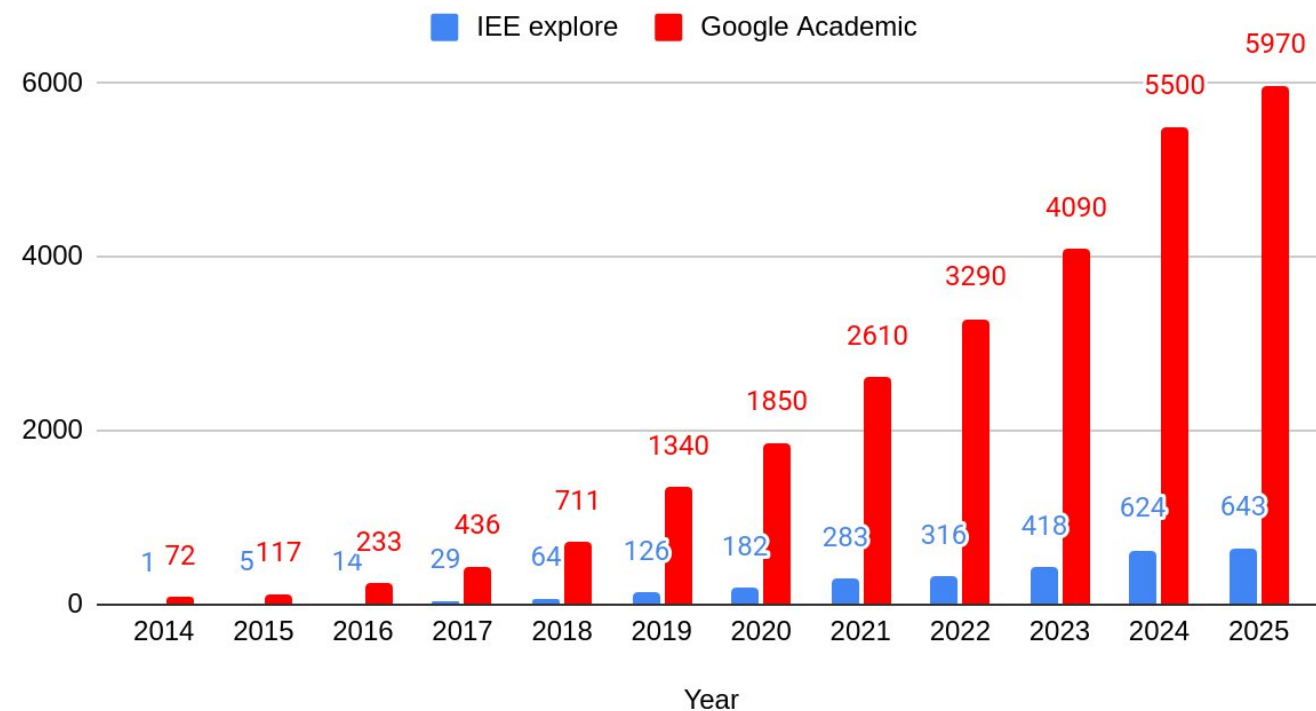


Example: UVM for a core



The RISC-V Revolution: Why It Matters for DV

IEE explore and Google Academic



RISC-V represents a paradigm shift in processor architecture through its open instruction set architecture. Unlike proprietary alternatives, RISC-V eliminates licensing fees and enables unprecedented customization, making it ideal for academic research, commercial products, and specialized applications.

The architecture's modular design philosophy allows implementers to select only needed extensions, reducing silicon complexity while maintaining compatibility. This modularity creates unique verification challenges that distinguish RISC-V DV from traditional processor verification workflows.

Origin	University of California, Berkeley
Governance	RISC-V Foundation
License	BSD (open and free)
Word size	32 bits, 64 bits, 128 bits (with SIMD or vector extensions)
Type	RISC, load/store
Base ISA	RV32I, RV64I, RV128I (fixed base, modular extensions)
Modular extensions	M (multiply and divide instructions) A (atomic operations) F (single-precision floating-point arithmetic) D (double-precision floating-point arithmetic) Q (quad-precision floating-point arithmetic) C (compressed instructions) B (bit manipulation) J (dynamically translated languages) P (SIMD instructions) V (vector instructions) E (embedded systems applications) G (general-purpose set: M, A, F, and D)
Register types	General-purpose registers (16 or 32, register x0 is hardwired to 0x0)
Instruction encoding	Variable-length
Branching	Comparison and branch instructions
Endianness	Little-endian
Modular ISA	Extensions can be added incrementally on top of the fixed base ISA

RISC-V

Modular Architecture

Ratified specs and extensions can be found at: <https://riscv.org/specifications/ratified/>

RISC-V Profiles: Standardized Configurations

RVA23 Profile

Application Processors

Targets rich OS environments like Linux, including virtual memory, atomics, and vector extensions. Defines minimum requirements for general-purpose computing workloads.

RVM23 Profile

Microcontroller Systems

Optimized for embedded systems with reduced complexity, focusing on deterministic interrupt latency and efficient code density for resource-constrained applications.

RVS23 Profile

Supervisor Domains

Enables secure partitioning and isolation for trusted execution environments, supporting hypervisor extensions and advanced security features.

Verification Implications of an Open ISA

The open nature of RISC-V introduces unique verification challenges that don't exist with proprietary architectures. Multiple vendors implement the same ISA specification with vastly different microarchitectures, each requiring tailored verification strategies.



Implementation Diversity

From in-order single-issue to out-of-order superscalar cores, further expanded by ISA configurability (e.g., VLEN, atomic reservation granularity), increasing verification complexity while preserving architectural correctness.



Extension Combinations

Exponential configuration space requires systematic coverage planning



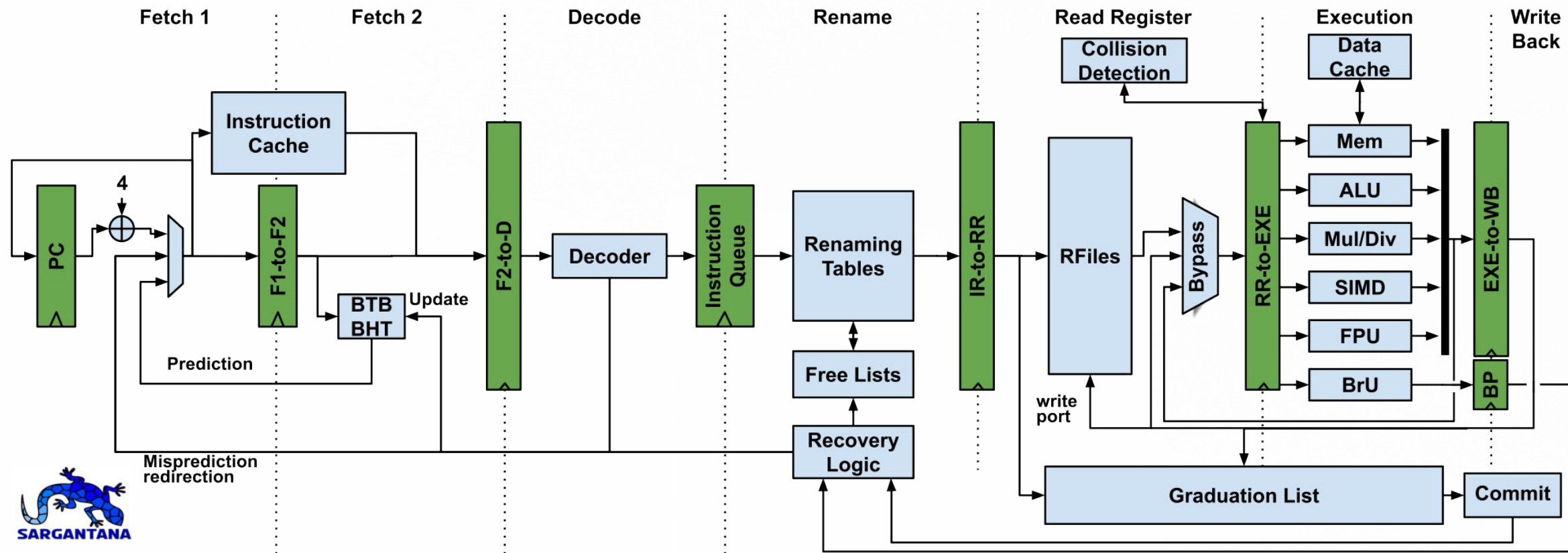
Compliance Testing

Formal conformance suites verify architectural correctness across implementations

DV engineers must develop portable verification IP that adapts to different implementations while ensuring ISA compliance, making RISC-V verification both challenging and intellectually rewarding.

Pipeline Implications for Verification

Understanding how instructions flow through pipeline stages is critical for effective verification. Each ISA extension introduces unique challenges:



- **Fetch:** Correct PC generation, instruction alignment, and branch prediction behavior
- **Decode:** Accurate opcode/format decoding and control signal generation
- **Rename:** Proper register renaming, dependency resolution, and recovery on mispredictions
- **Register Read:** Operand availability, stall handling, and forwarding correctness
- **Execute:** Functional correctness, exception detection, and latency handling
- **Write Back:** Correct result commit, register state update, and in-order retirement

Verification Challenges per Extension

Atomic Operations (A Extension)

The A extension presents complex verification scenarios requiring multi-threaded test sequences. Key challenges include:

- Verifying atomicity guarantees under concurrent access
- Testing LR/SC reservation set behavior and spurious failures
- Validating AMO ordering with respect to other memory operations
- Checking forward progress guarantees

Compressed Instructions (C Extension)

The C extension creates decode complexity requiring careful verification:

- Correct expansion to 32-bit equivalents
- Mixed 16/32-bit instruction alignment handling
- Branch target address calculation with compressed instructions
- Compressed instruction exception behavior

Custom Extensions: Extending the ISA

RISC-V reserves opcode space (custom-0 through custom-3) for implementer-specific extensions without fragmenting the ISA ecosystem. This enables domain-specific acceleration while maintaining base ISA compatibility.

Opcode Allocation

Four major opcodes reserved for custom instructions with encoding flexibility

Verification Strategy

Custom extensions require specialized testbenches and reference models for functional validation

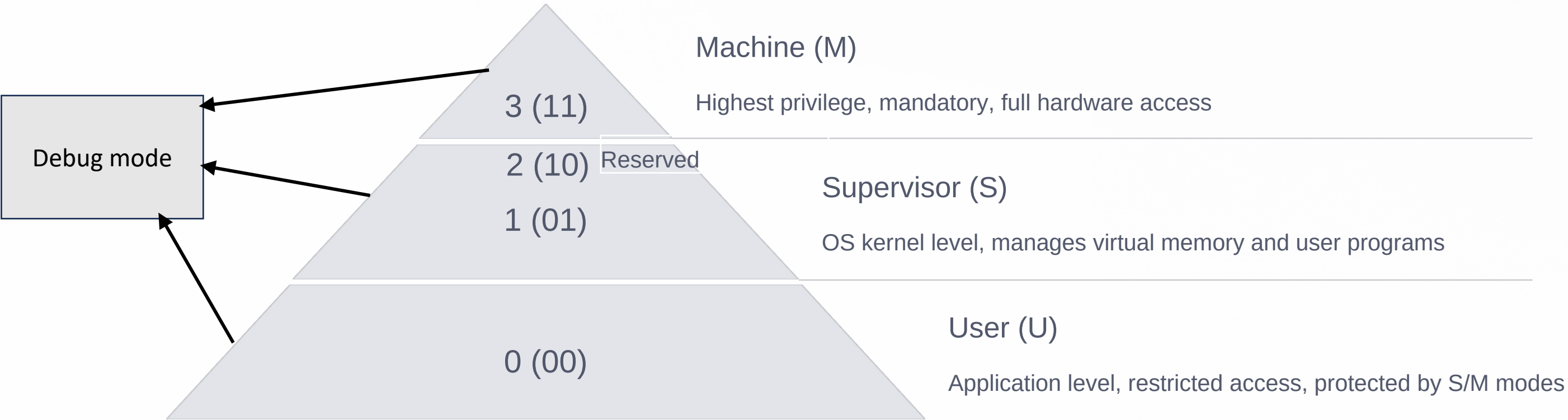
Tool Integration

Extending Spike and assemblers to support custom instructions for comprehensive verification

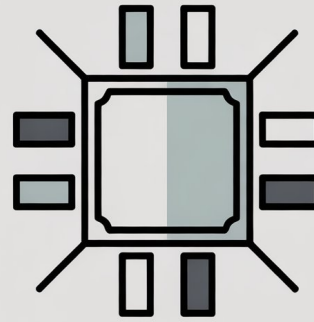
inst[4:2] inst[6:5]	000	001	010	011	100	101	110	111 (>32b)
00	LOAD	LOAD-FP	custom-0	MISC-MEM	OP-IMM	AUIPC	OP-IMM-32	48b
01	STORE	STORE-FP	custom-1	AMO	OP	LUI	OP-32	64b
10	MADD	MSUB	NMSUB	NMADD	OP-FP	OP-V	custom-2/rv128	48b
11	BRANCH	JALR	reserved	JAL	SYSTEM	OP-VE	custom-3/rv128	≥80b

RISC-V Privilege Levels

RISC-V defines a hierarchical privilege architecture enabling secure separation between system software, OS kernel, and user applications. This three-level model provides flexibility for implementations ranging from simple embedded systems to complex virtualized servers.



Additional privilege levels (HS, VS) support virtualization extensions, enabling hypervisor-based systems with nested privilege domains.



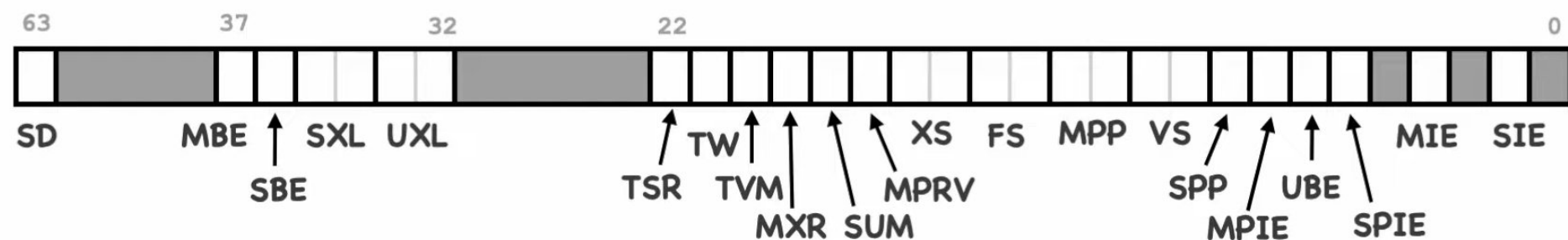
Control and Status Registers (CSRs)

CSRs provide the interface for privileged operations, system configuration, and exception handling. The 12-bit CSR address space encodes access privileges in bits [11:10], enabling hardware enforcement of security policies.

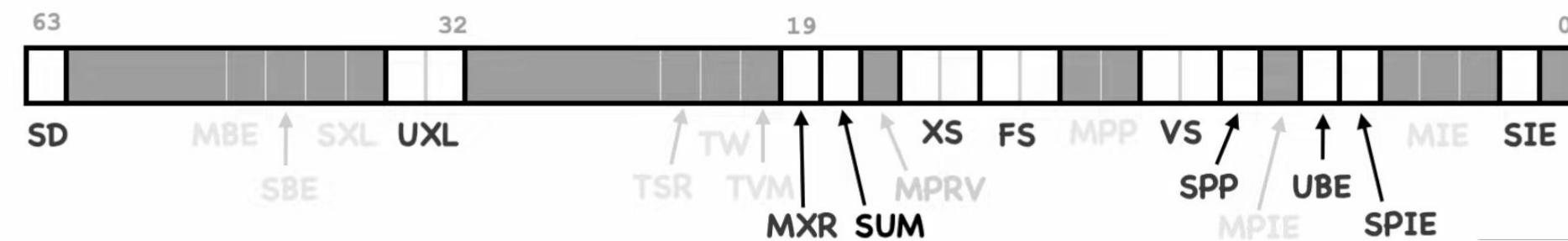
Key CSR categories include machine-mode control (mstatus, mie, mtvec), supervisor-mode state (sstatus, sie, stvec), performance counters, and architectural state registers. Each CSR has defined read/write semantics with specific fields that may be WLRL (Write Legal, Read Legal) or WARL (Write Any, Read Legal).

Critical Machine-Mode CSRs RV32

mstatus



sstatus

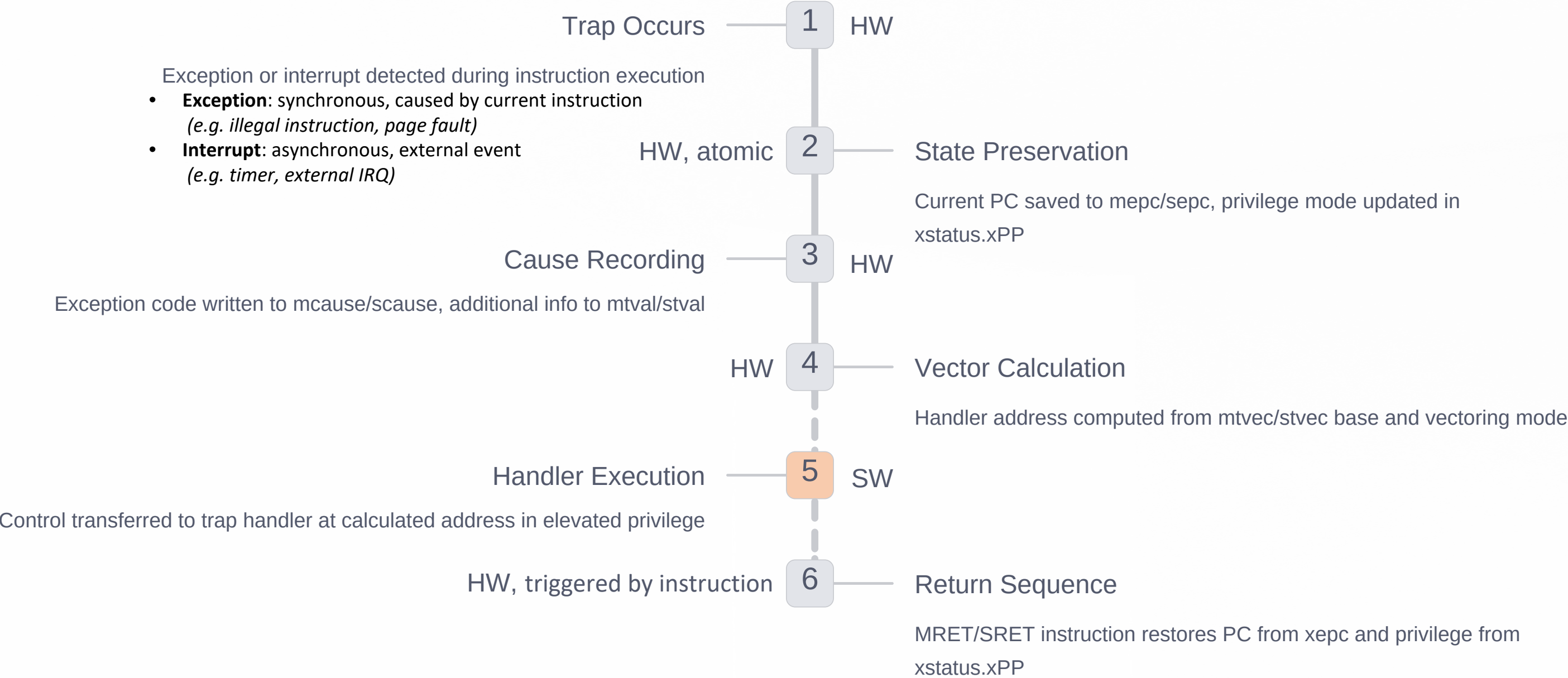


- SXL, UXL – Modify XLEN
- MBE, SBE, UBE – Big Endian
- MPRV – Modify Privilege Level
- SUM – Supervisor User Memory
- MXR – Make Executable Readable
- FS, VS, XS – Need to save additional thread state
- SD – Summary Dirty
- TSR – Trap SRET
- TVM – Trap Virtual Memory
- TW – Timeout Wait
- MPP, SPP – Previous Privilege Mode
- MPIE, SPIE – Previous Interrupt Enable
- MIE, SIE – Interrupt Enable

Exception and Interrupt Handling Flow

Exception or interrupt detected during instruction execution

- **Exception:** synchronous, caused by current instruction
(e.g. illegal instruction, page fault)
- **Interrupt:** asynchronous, external event
(e.g. timer, external IRQ)



Trap Delegation Mechanism

Selective Delegation

The medeleg and mideleg CSRs enable machine mode to delegate specific exception and interrupt types to supervisor mode, reducing trap handling latency for OS operations.

Each bit in medeleg/mideleg corresponds to an exception/interrupt code. When set, matching traps transfer control to supervisor handlers (stvec) rather than machine handlers (mtvec), preserving M-mode for critical system management.

Code	Hex	Mask	Abbr	Description
0	0	000001		
1	1	000002	SSI	Supervisor software interrupt
2	2	000004	VSSI	Virtual supervisor software interrupt
3	3	000008	MSI	Machine software interrupt
4	4	000010		
5	5	000020	STI	Supervisor timer interrupt
6	6	000040	VSTI	Virtual supervisor timer interrupt
7	7	000080	MTI	Machine timer interrupt
8	8	000100		
9	9	000200	SEI	Supervisor external interrupt
10	a	000400	VSEI	Virtual supervisor external interrupt
11	b	000800	MEI	Machine external interrupt
12	c	001000	SGEI	Supervisor guest external interrupt
13	d	002000	LCOFI	Local counter overflow interrupt
14	e	004000		
15	f	008000		

Code	Hex	Mask	Description
0	0	000001	Instruction address misaligned
1	1	000002	Instruction access fault
2	2	000004	Illegal instruction
3	3	000008	Breakpoint
4	4	000010	Load address misaligned
5	5	000020	Load access fault
6	6	000040	Store/AMO address misaligned
7	7	000080	Store/AMO access fault
8	8	000100	Environment call from U-mode or VU-mode
9	9	000200	Environment call from HS-mode
10	a	000400	Environment call from VS-mode
11	b	000800	Environment call from M-mode
12	c	001000	Instruction page fault
13	d	002000	Load page fault
14	e	004000	
15	f	008000	Store/AMO page fault
16	10	010000	Double trap
17	11	020000	
18	12	040000	Software check
19	13	080000	Hardware error
20	14	100000	Instruction guest-page fault
21	15	200000	Load guest-page fault
22	16	400000	Virtual instruction
23	17	800000	Store/AMO guest-page fault

Verification Scenarios for CLINT/PLIC



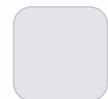
Register Access Validation

Test all CLINT/PLIC registers for correct read/write semantics, reserved field handling, and access permission enforcement across privilege modes



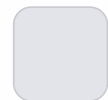
Trap Latency Measurement

Measure cycles from interrupt assertion to handler entry, ensuring deterministic bounds and priority-based preemption behavior



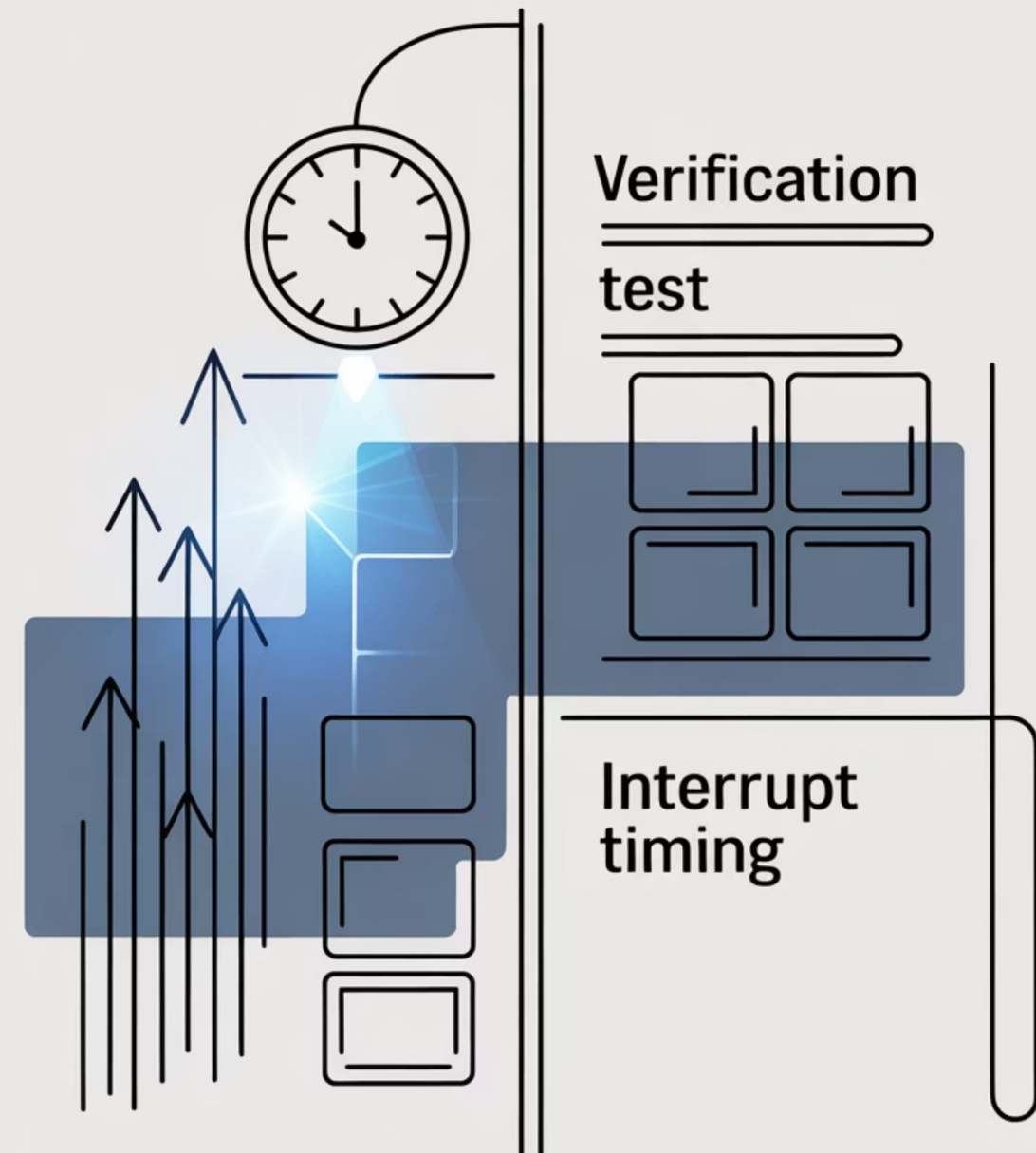
Priority and Context Checking

Verify correct priority arbitration with simultaneous interrupts, threshold masking, and routing to appropriate hart contexts



Edge Case Coverage

Test claim/complete race conditions, rapid enable/disable transitions, and interrupt storm scenarios with maximum source assertion



The V extension: Understanding VLEN, SEW, and LMUL

VLEN

Vector Register Length

Implementation-specific parameter defining physical width of each vector register in bits. Common values: 128, 256, 512, 1024. Fixed per implementation but portable code adapts automatically.

SEW

Selected Element Width

Configurable element size: 8, 16, 32, or 64 bits. Set by vtype CSR. Determines how many elements fit in VLEN bits. Example: VLEN=256, SEW=32 → 8 elements per register.

LMUL

Length Multiplier

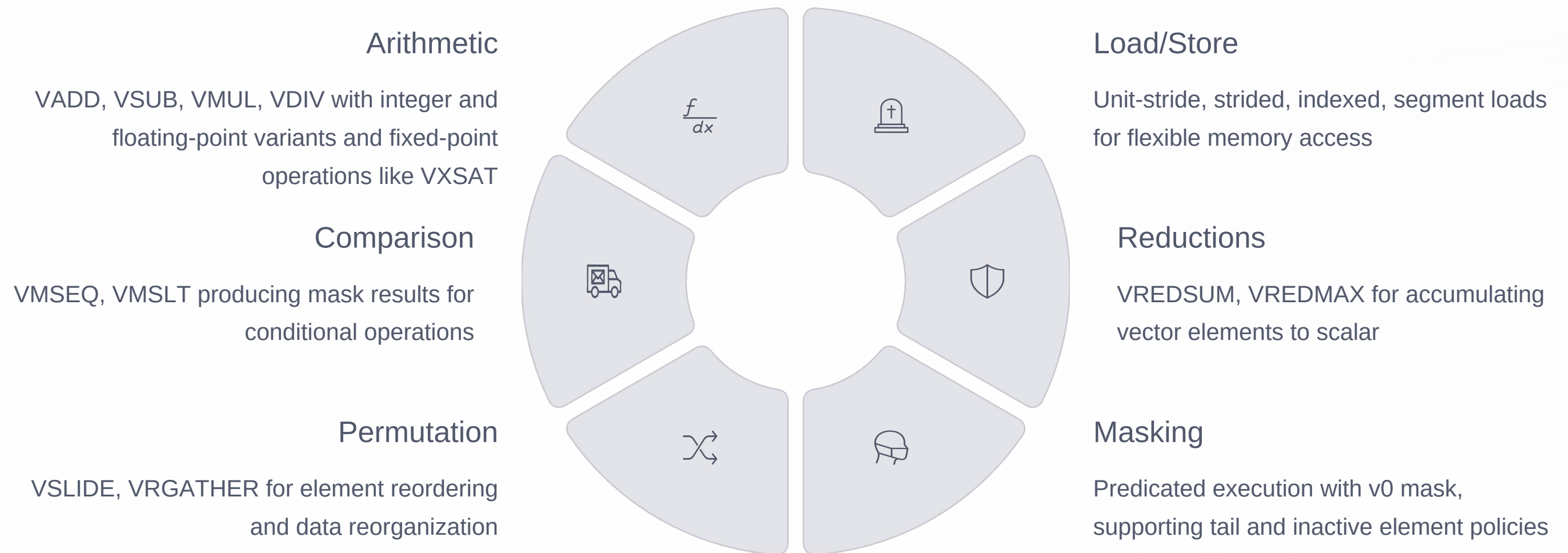
LMUL (1/8 to 8) groups registers to scale vector length. Values > 1 combine registers to process more elements per instruction, while values < 1 use register fractions to handle mixed-width data efficiently.

VL

Vector Length

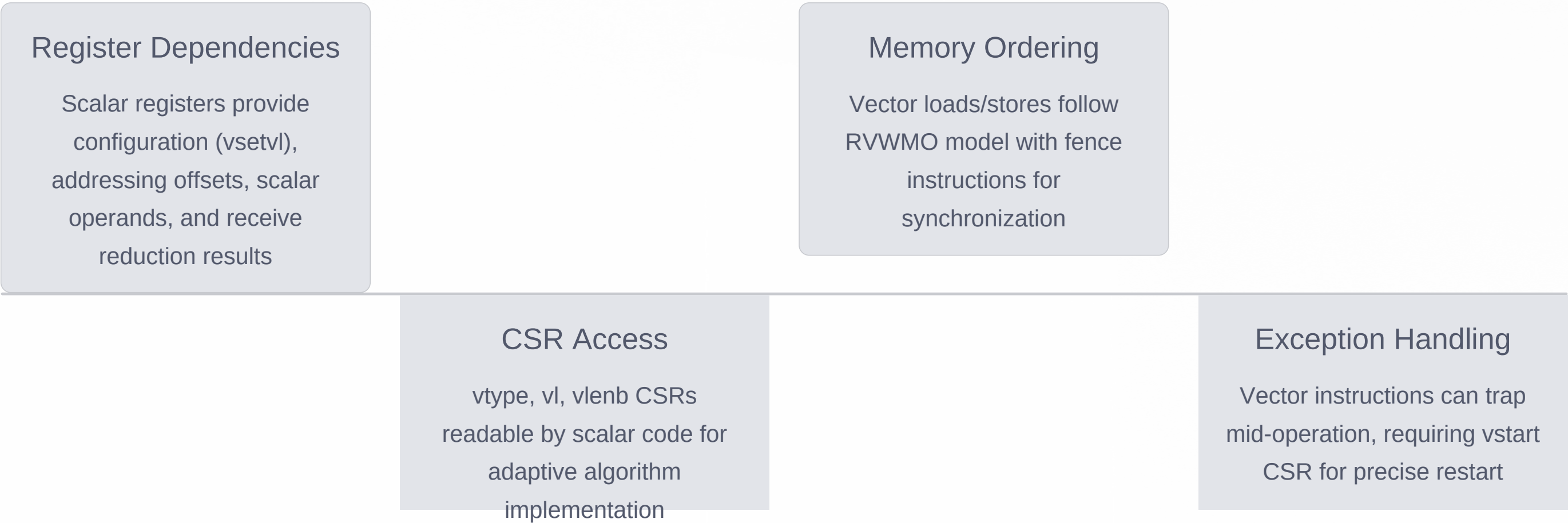
Number of active elements for current operation, set by vsetvl instruction. $VL \leq VLMAX (VLEN \times LMUL / SEW)$. Handles arbitrary problem sizes with tail processing.

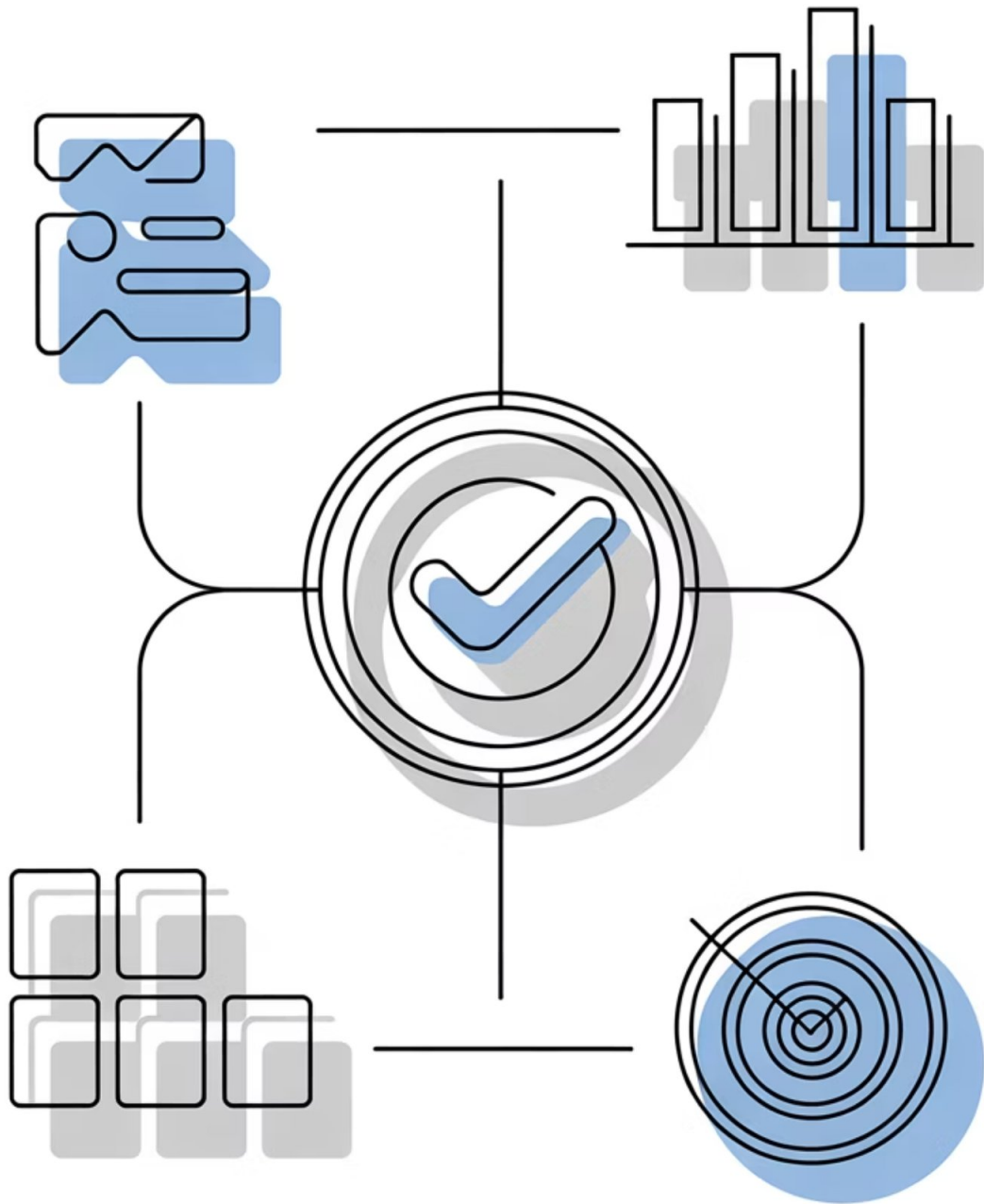
Vector Instruction Classes



Scalar-Vector Pipeline Interactions

Vector operations interact with scalar pipeline through register dependencies, CSR access, and memory system. Understanding these interactions is critical for performance and correctness verification.





Verification Strategies for RVV

Configuration Space

Randomize vtype settings (SEW, LMUL) and VL across full valid range in every test

Cross-Checking

Compare RTL results against Spike/RISCV-VP reference models for arithmetic correctness

Corner Cases

Test VL=0, maximum VLMAX, LMUL fractional values, and illegal vtype configurations

Functional Coverage

Cover all instruction variants, masking modes, element widths, and register group combinations

Physical vs Virtual Memory

Physical Memory

Direct hardware addressing of DRAM and memory-mapped I/O.
Physical addresses represent actual locations in the memory system.
Used in embedded systems and by machine-mode software.

Advantages:

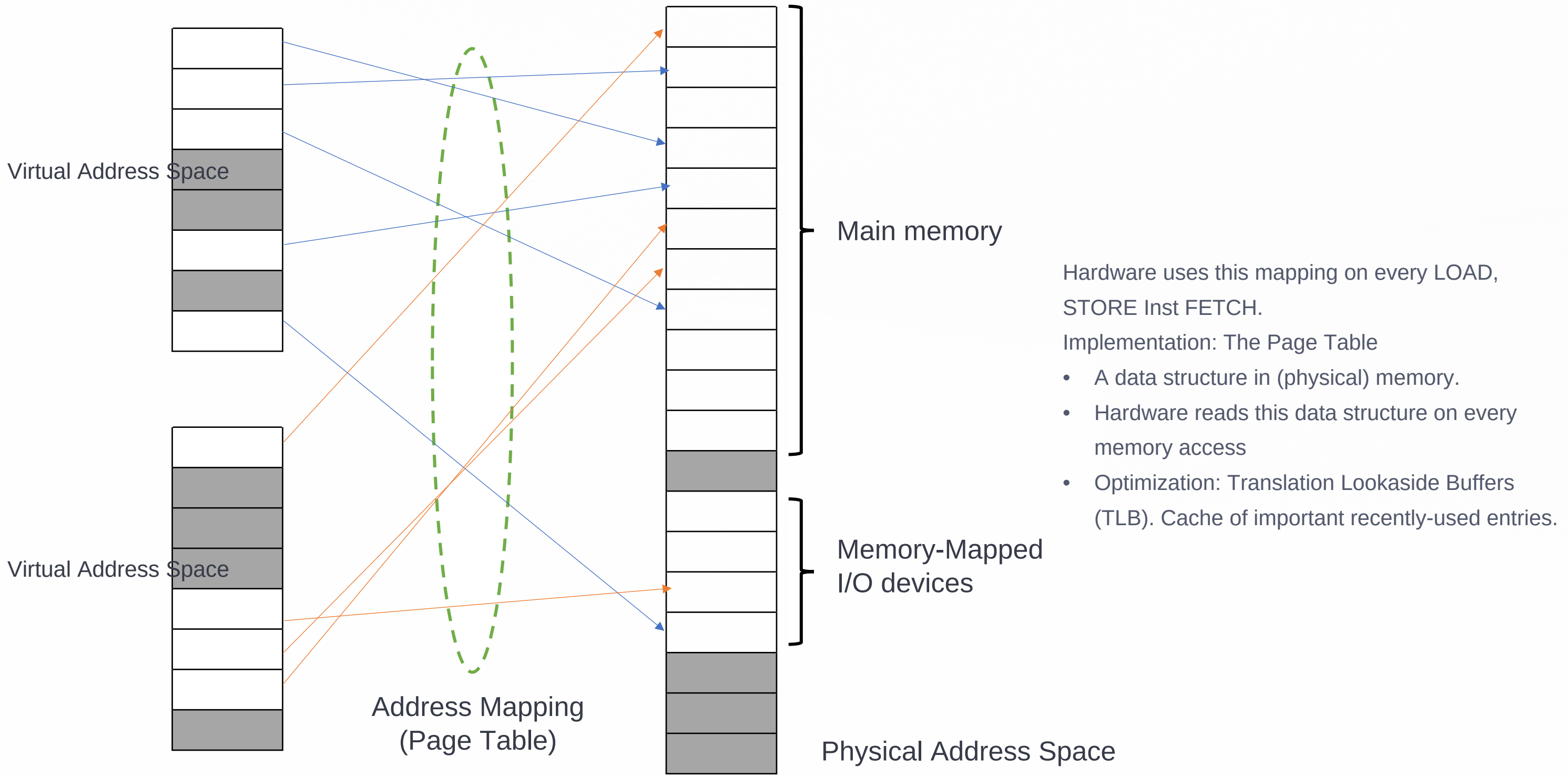
- Simple, direct access with minimal overhead
- Deterministic latency for real-time systems
- Full memory map control

Virtual Memory

Abstraction layer enabling memory protection, isolation, and address space management. Virtual addresses translated through page tables to physical addresses.

Advantages:

- Process isolation and protection
- Demand paging and memory overcommit
- Simplified application memory model
- Efficient memory sharing between processes



Address Translation Walk Sequence

Extract VPN Levels

Parse virtual address into VPN[1] and VPN[0] based on page table scheme (Sv32 shown)

Read Root PTE

Calculate address: $\text{satp.PPN} \times \text{PAGESIZE} + \text{VPN}[1] \times 4$

Read 32-bit PTE from memory

Check Validity

Verify V=1, check permission bits (R,W,X)

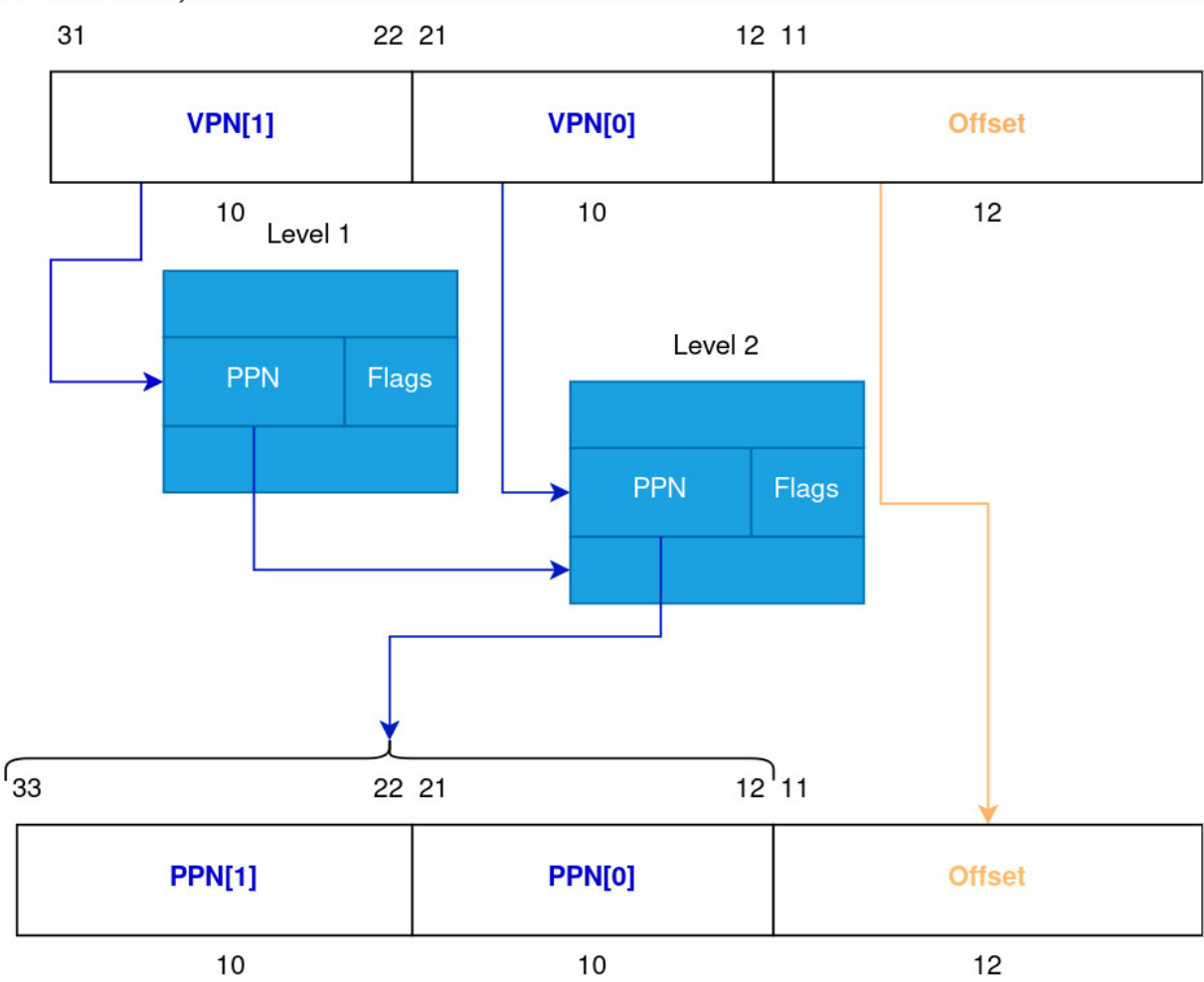
Raise page fault if invalid or insufficient permissions

Recurse or Translate

If R=0 and W=0 and X=0, PTE points to next level;
otherwise leaf PTE, construct physical address

Update Access Bits

Set A (accessed) bit, set D (dirty) bit for stores, cache translation in TLB



Page Faults and Exception Handling

Page faults occur when address translation fails or access permissions are violated. Proper exception handling enables demand paging and memory protection enforcement.

Instruction Page Fault

Exception code 12, triggered by invalid instruction fetch translation, stval contains faulting address

Load Page Fault

Exception code 13, load operation with invalid translation or insufficient read permission

Store Page Fault

Exception code 15, store operation with invalid translation or insufficient write permission

OS page fault handlers examine stval and cause registers, potentially loading pages from disk, adjusting permissions, or terminating processes for genuine violations.

Verification Implications for Virtual Memory



Page Table Construction

Generate valid multi-level page tables with diverse configurations: different page sizes, permission combinations, and address space layouts. Test invalid PTE formats for exception handling.



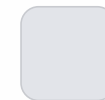
Translation Correctness

Compare RTL address translation against Spike reference for thousands of random virtual addresses across all page table levels and page sizes.



Permission Enforcement

Verify read/write/execute permissions are correctly enforced, generating appropriate page faults for violations. Test user/supervisor mode interaction.



TLB Coherence

Ensure SFENCE.VMA correctly invalidates translations. Test race conditions between TLB updates and page table modifications.

Consistency Models: The Landscape

Sequential Consistency (SC)

The most intuitive model where all operations appear to execute one at a time in a global total order that respects program order. Strict, predictable, but limits hardware optimization performance.

Total Store Order (TSO)

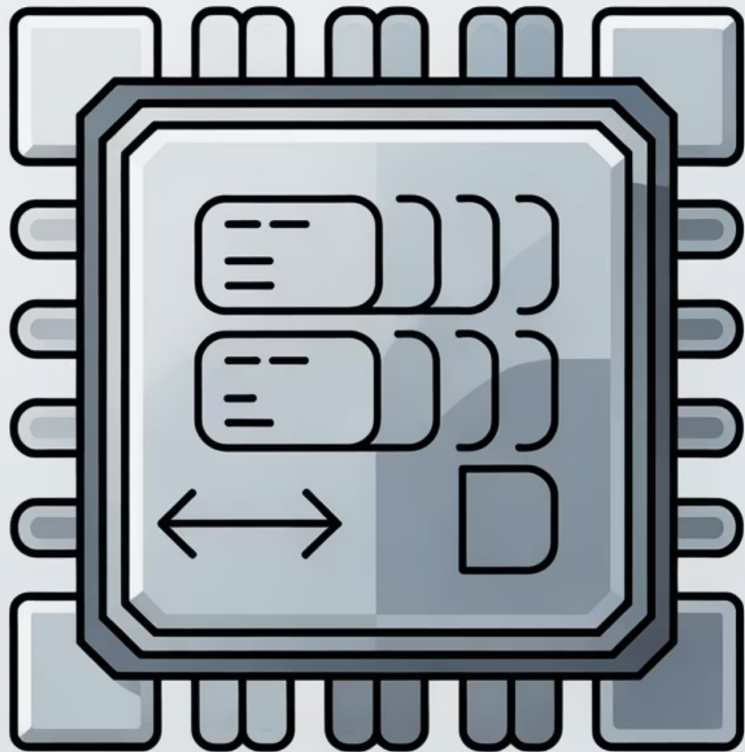
Used by x86. Writes can be buffered (store buffer), allowing reads to bypass earlier writes to different addresses. Still relatively strong but offers better performance than SC.

RISC-V Weak Memory Ordering (RVWMO)

Designed for maximum hardware flexibility and performance. Allows extensive reordering of loads and stores unless explicitly constrained by fences or dependencies. It is "weaker" than TSO.

As verification engineers, moving from a TSO mindset (common in x86) to RVWMO requires a fundamental shift. You cannot assume that if Core A writes Data then Flag, Core B will see Flag then Data, unless explicit synchronization is used.

Why Weak Ordering Matters for DV



Hardware designers utilize weak memory models to optimize pipeline performance. They implement aggressive load-store units (LSUs) that reorder memory accesses to hide latency.

The Verification Challenge

RTL bugs often lurk in the corner cases of these reorderings. A valid RISC-V core **must** legally reorder instructions according to RVWMO rules. If your DV environment assumes Sequential Consistency, you will miss critical bugs or flag valid hardware behaviors as errors.

- Observability: Different harts (hardware threads) may observe memory operations in different orders.
- Atomicity: Load-Reserved/Store-Conditional (LR/SC) sequences require precise checking.

Enforcing Order: FENCE Instructions

To regain ordering when necessary (e.g., driver logic, synchronization locks), RISC-V provides explicit fence instructions.



FENCE

The standard ``fence`` instruction orders device I/O and memory accesses. It takes predecessor (P) and successor (S) arguments (R/W/I/O) to specify exactly what operations must complete before others begin. `fence rw, rw` (often just ``fence``) ensures all previous loads/stores complete before subsequent ones.



FENCE.I

Explicitly synchronizes the instruction stream with data memory. Essential for self-modifying code or when loading programs into RAM. It ensures that stores to instruction memory are visible to instruction fetches.

Preserved Program Order (PPO)

Even in a weak model, the hardware must respect data dependencies to ensure a single thread executes logically. RVWMO defines "Preserved Program Order" rules.

Address Dependency

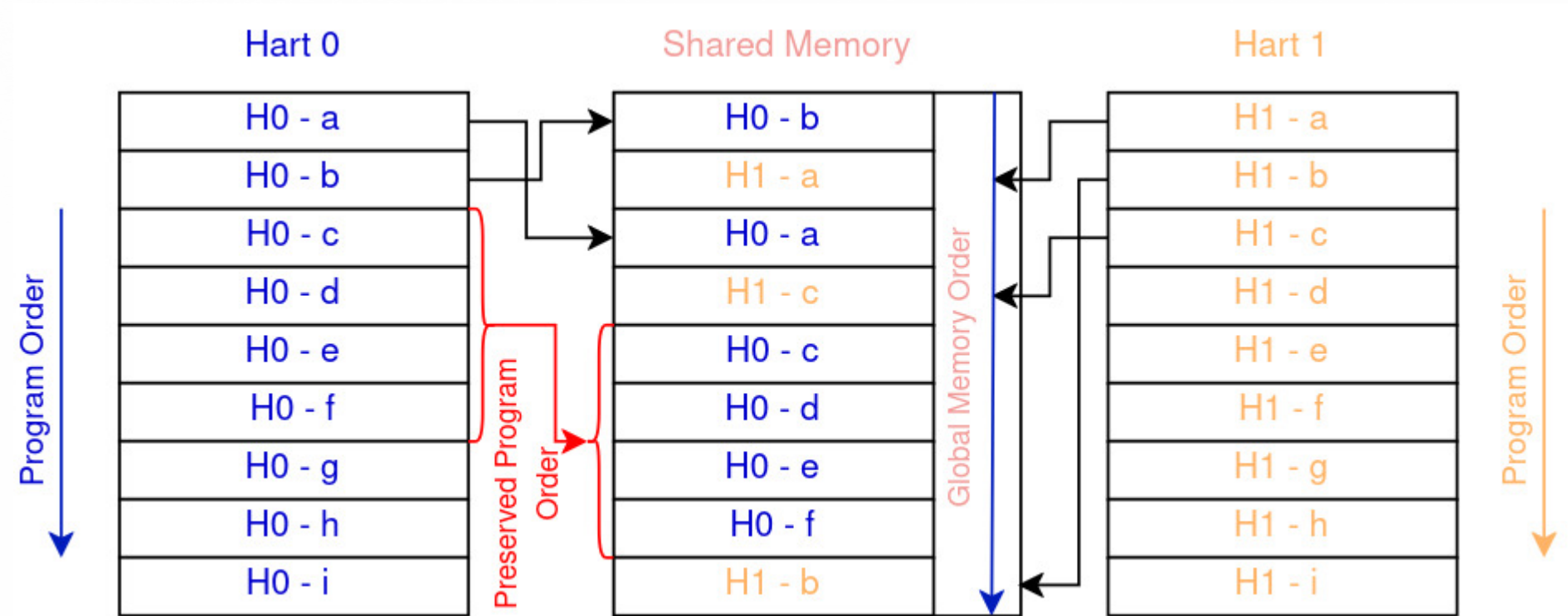
If instruction B uses the address computed by instruction A, B cannot execute before A.

Data Dependency

If instruction B writes data computed by A, or reads data written by A (to the same register), ordering is preserved.

Control Dependency

Branch decisions must resolve before subsequent memory operations are globally visible (with some exceptions for control-dependent stores).



Multi-Core Implications & Litmus Tests

The classic way to verify memory consistency is through "Litmus Tests"—small assembly programs designed to expose specific ordering behaviors.

```
// Message Passing Example
// Core 0
store x, 1 // (a)
fence w, w // Enforce order
store y, 1 // (b)

// Core 1
load r1, y // (c)
fence r, r // Enforce order
load r2, x // (d)
```

Scenario: Without fences, hardware could reorder (a) and (b), or (c) and (d).

Outcome: If `r1` reads `1` (sees the flag), can `r2` read `0` (miss the data)?

Under RVWMO, without the fences, this outcome is **allowed**. With the fences, it is **forbidden**. Verification must automatically check thousands of these permutations.

Verification Strategy: Litmus Testing

How do we scale this verification? We cannot manually write every test.



Test Generation

Use tools like the `herd7` toolchain or custom generators to produce thousands of varying litmus tests covering different hazard scenarios and fence combinations.



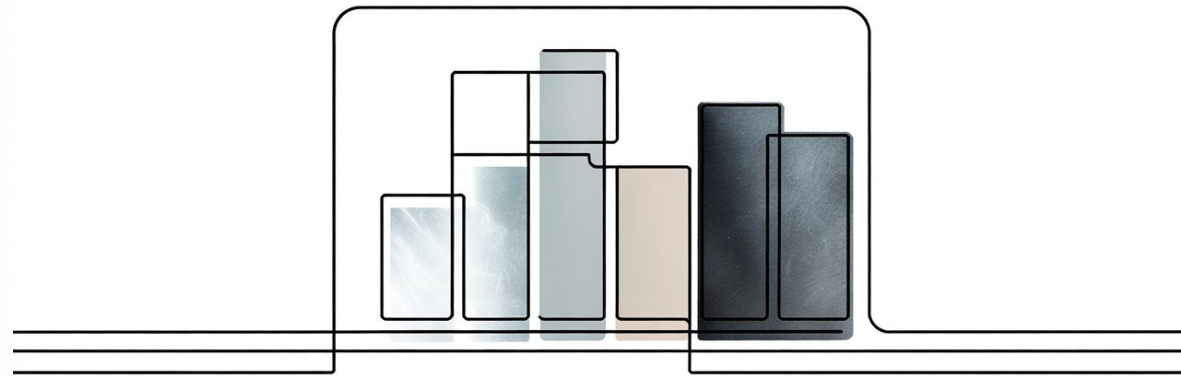
RTL Simulation

Run these tests on the DUT (Device Under Test). The simulation must involve pseudo-random delays in memory interfaces (e.g., AXI/TileLink) to stress-test the reordering logic.



Outcome Analysis

Compare the final register states against the "allowed behaviors" defined by the formal RVWMO model. Any behavior observed in RTL that is forbidden by the model is a bug.



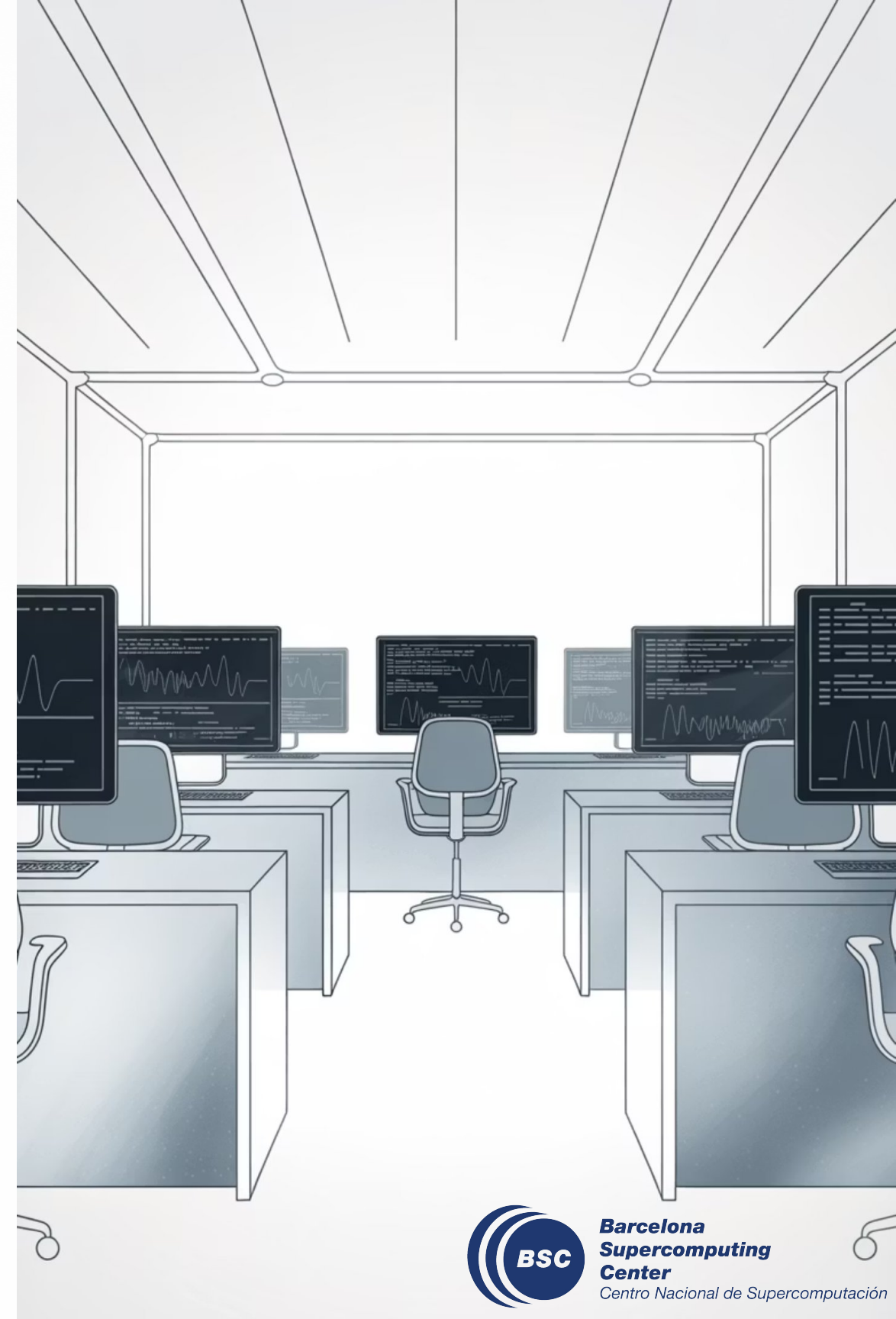
DV Scoreboard Considerations for RVWMO

A standard in-order scoreboard will fail in a multi-core RVWMO environment. Your scoreboard must be "memory model aware."

- **Relaxed Checking:** The scoreboard cannot simply expect strict global ordering. It must track valid partial orderings.
- **Tilted checking:** Often, scoreboards compare against an ISS (Instruction Set Simulator) like Spike. Standard Spike runs in SC (Sequential Consistency) mode. You may need to configure Spike to emulate weak ordering or use a more advanced golden reference that supports concurrency.
- **Snooping:** The verification environment (UVM agent) needs visibility into the cache coherence traffic to validate that the coherence protocol (e.g., MESI) is correctly supporting the memory model.

DV Methodology for RISC-V

Moving beyond the ISA theory, we focus now on the practical application of Verification methodologies. We will construct a robust UVM environment, integrate standard RISC-V reference models, and define a strategy for regression and coverage closure using industry-standard tools.



**Barcelona
Supercomputing
Center**
Centro Nacional de Supercomputación

Step-and-Compare Flow

1. Program Generation

RISC-V programs are generated using either **constrained-random instruction generation** or **directed tests**, depending on the verification objectives.

2. Compilation & ELF Generation

The generated programs are compiled to produce a RISC-V ELF binary.

3. Testbench Initialization

The ELF is loaded into both the DUT memory and the reference model memory.

4. Lockstep Execution (Step)

The DUT and the reference simulator (Spike) execute the program instruction by instruction in lockstep.

5. State Comparison (Compare)

After each instruction, architectural states (PC, registers, CSRs, and memory) are compared.

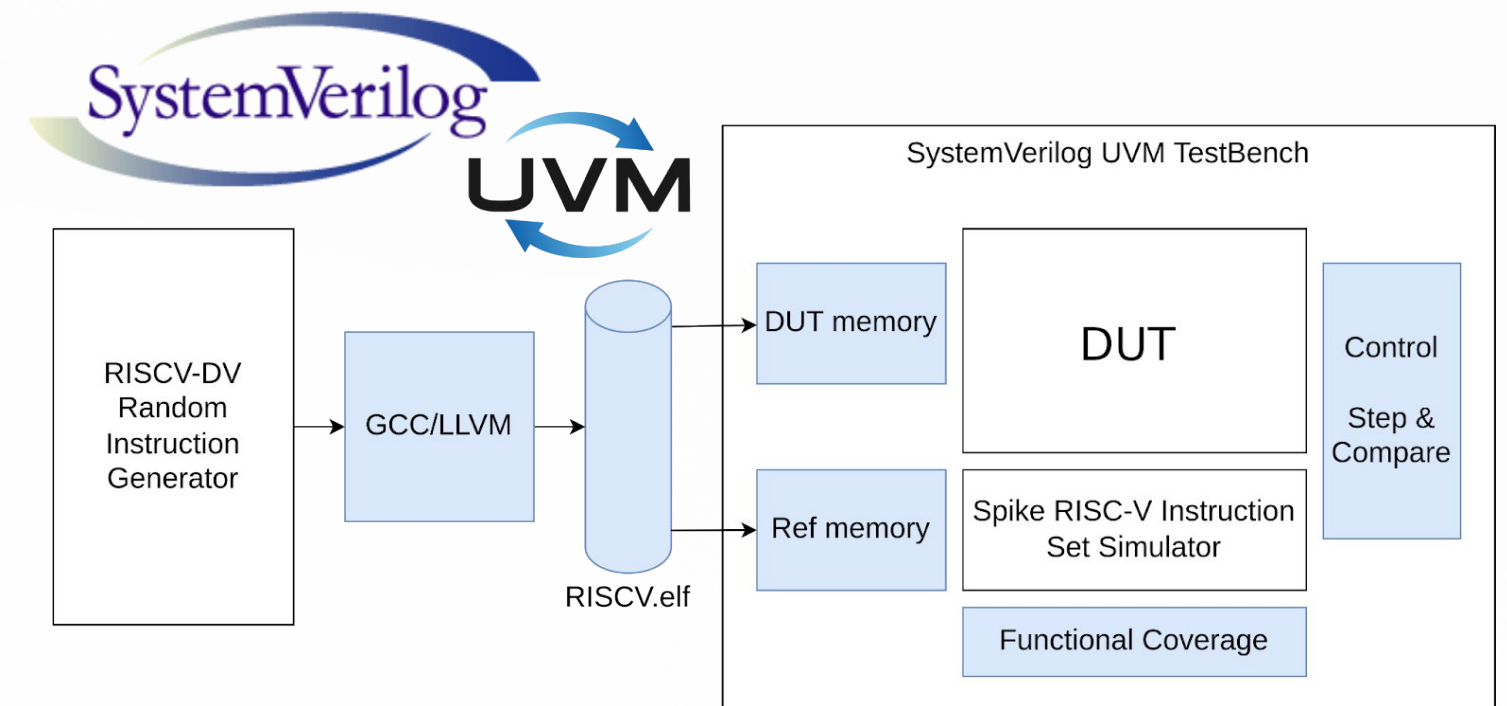
6. Error Detection

Any mismatch between DUT and reference state is reported as a functional failure.

7. Functional Coverage Collection

Coverage metrics are collected during execution to measure verification completeness.

The most reliable way to verify a processor core is **co-simulation**.



The Golden Reference: Spike ISS

The foundation of RISC-V verification is the Instruction Set Simulator (ISS). Spike is the official RISC-V golden reference model.

What is Spike?

Spike simulates a RISC-V processor functionally. It executes instructions and updates architectural state (PC, GPRs, CSRs) instantly, without modeling microarchitectural timing or pipelines.

DV Integration Role

We use Spike as a predictor. By feeding the same instruction stream to both the RTL and Spike, we can compare the resulting state after every instruction retirement. This "Step-and-Compare" methodology is the industry standard for CPU verification.



Configurability

Spike must be configured (via ISA string arguments) to match your specific RTL features (e.g., ``rv64gcv``, specific CSR values).

Random Instruction Generation: RISC-V-DV

Directed tests are insufficient for complex processors. We need constrained random generation to hit corner cases. Google's **RISC-V-DV** is the open-source standard framework for this.



Generator

SystemVerilog-based generator creates a stream of assembly instructions. It uses constraints to bias towards specific scenarios (e.g., "lots of arithmetic," "heavy branching," "illegal instructions").



Templates

Users write "instruction stream templates" rather than fixed assembly. The generator solves the constraints to produce valid, randomized assembly programs.



Compilation

The generated assembly is compiled using GCC into an ELF binary, which is then loaded into the RTL and Spike simulations.

Compliance Testing Suites

Before running random stress tests, you must ensure basic architectural compliance.

riscv-arch-test

An official repository of "architectural tests." These are static assembly tests designed to check that an implementation complies with the ISA specification.

- Checks basic instruction functionality (ADD, SUB, JAL).
- Verifies specific extensions (Compressed instructions, Vector extension).
- Result is a "signature" file that is compared against a golden reference signature.

<https://github.com/riscv-non-isa/riscv-arch-test>

riscv-isa-tests

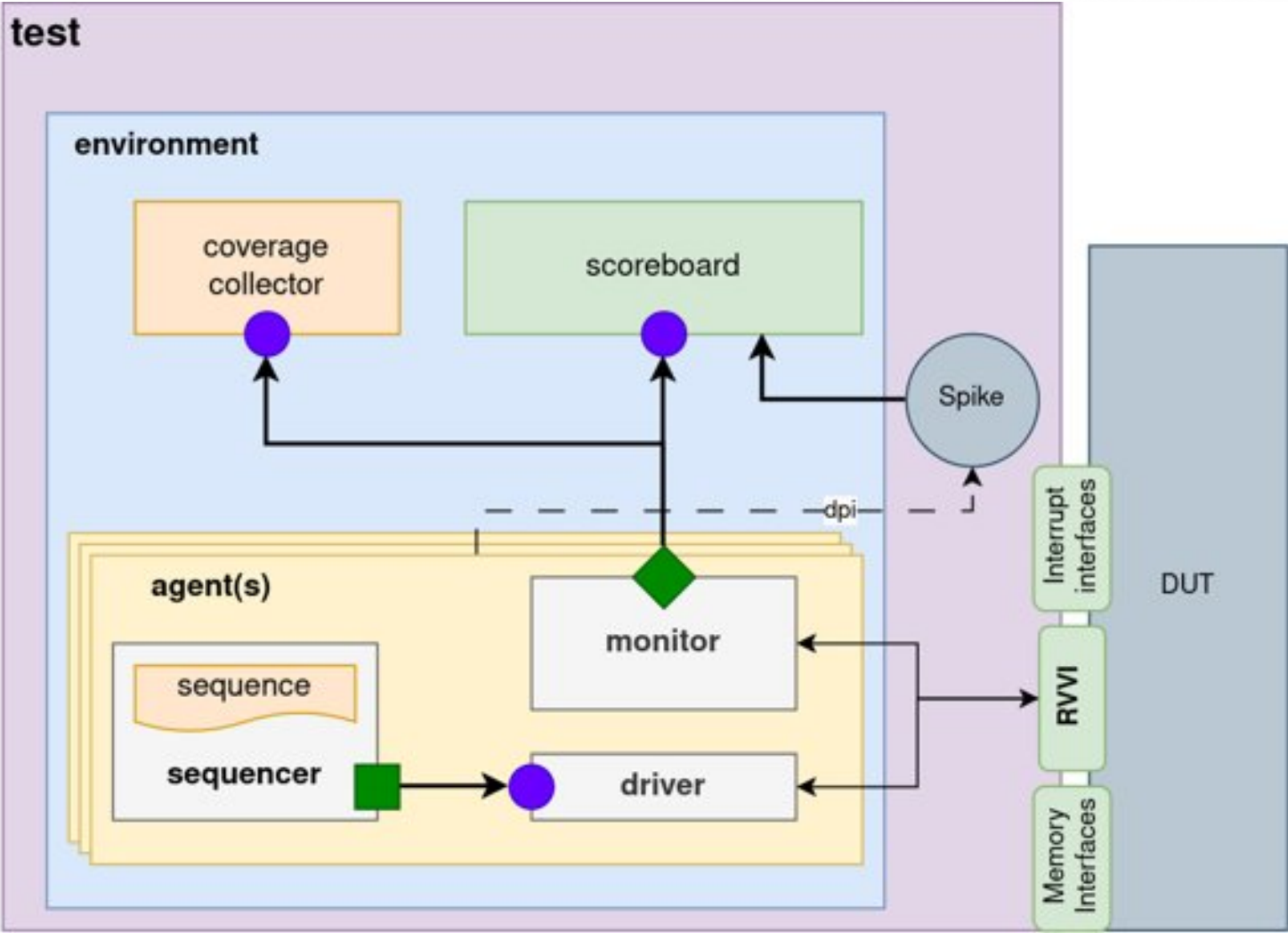
The older, classic suite of tests often included in toolchains. While still useful for sanity checking, `riscv-arch-test` provides more rigorous coverage of the specification.

Usage Strategy: Run these as sanity checks (smoke tests) on every RTL commit. If these fail, there is no point in running complex UVM regressions.

<https://github.com/riscv-software-src/riscv-tests>

UVM Environment Architecture

A robust RISC-V UVM environment is complex. It interacts with memory, interrupts, and debug interfaces.



The Agents

Instruction Agent: Monitors the instruction retirement interface (often a proprietary trace interface or the standard RVVI). **Memory Agent:** Models the AXI/TileLink bus, acting as memory and responding to load/store requests. **Interrupt Agent:** Drives asynchronous interrupt lines to stress exception handling.

The Scoreboard

The heart of the environment. It receives the "retired instruction" transaction from the Instruction Agent and compares it against the predicted state from the Reference Model (Spike). It must handle non-deterministic events like interrupts by synchronizing the reference model with the RTL's interrupt timing.

Handling Asynchronous Events

Verifying synchronous functionality is easy; verifying interrupts is hard.

"When an interrupt asserts, the pipeline doesn't stop immediately. It might retire 2-3 more instructions before jumping to the trap handler. How does the Golden Model know exactly when to jump?"

The Solution: Interactive Co-simulation. Instead of running Spike completely independently, the UVM Scoreboard often "guides" Spike. When the RTL takes a trap, the Scoreboard tells Spike: "Force a trap now." Then, we verify that the RTL jumped to the correct vector address and updated the ``mcause`` and ``mepc`` CSRs correctly.

Functional Coverage Strategy

How do you know you are done? Code coverage is not enough. You need Functional Coverage models mapped to the ISA.

Instruction Cross-Coverage

We need to observe interactions between instructions, not just the instructions themselves.

- **Hazards:** RAW, WAW, WAR dependencies between adjacent instructions.
- **ALU-to-Branch:** Arithmetic operation followed immediately by a branch dependent on the result.
- **Memory-to-Memory:** Back-to-back load/store operations to test LSU buffering.

Privileged Mode Coverage

Coverage groups specifically for system-level behavior.

- **CSR Transitions:** Writing all valid values to all CSRs.
- **Exception Crosses:** Triggering every type of exception (illegal inst, misalignment, ecall) from every privilege mode (User, Supervisor, Machine).

Extending to Vectors and Custom Extensions

RISC-V's modularity means your verification environment must be extensible.

Vector Extension (RVV)

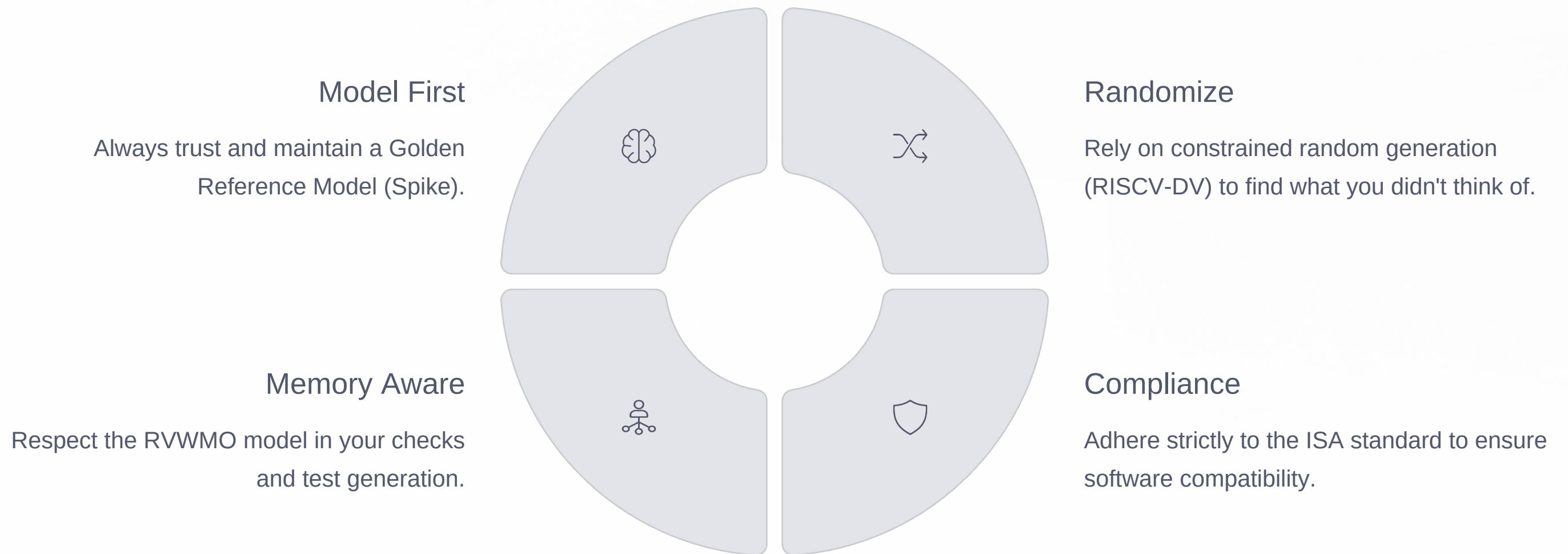
Verifying RVV is massive. The vector length (VLEN) is parameterizable. Your UVM agents must support dynamic array comparisons and your random generator must create heavy data-parallel workloads to stress the vector engine.

Custom Instructions

Many companies add custom accelerators. You must extend Spike (C++ model) to support these new opcodes so the scoreboard can predict them. Never verify custom logic with RTL-only self-checking; always update the Golden Model.

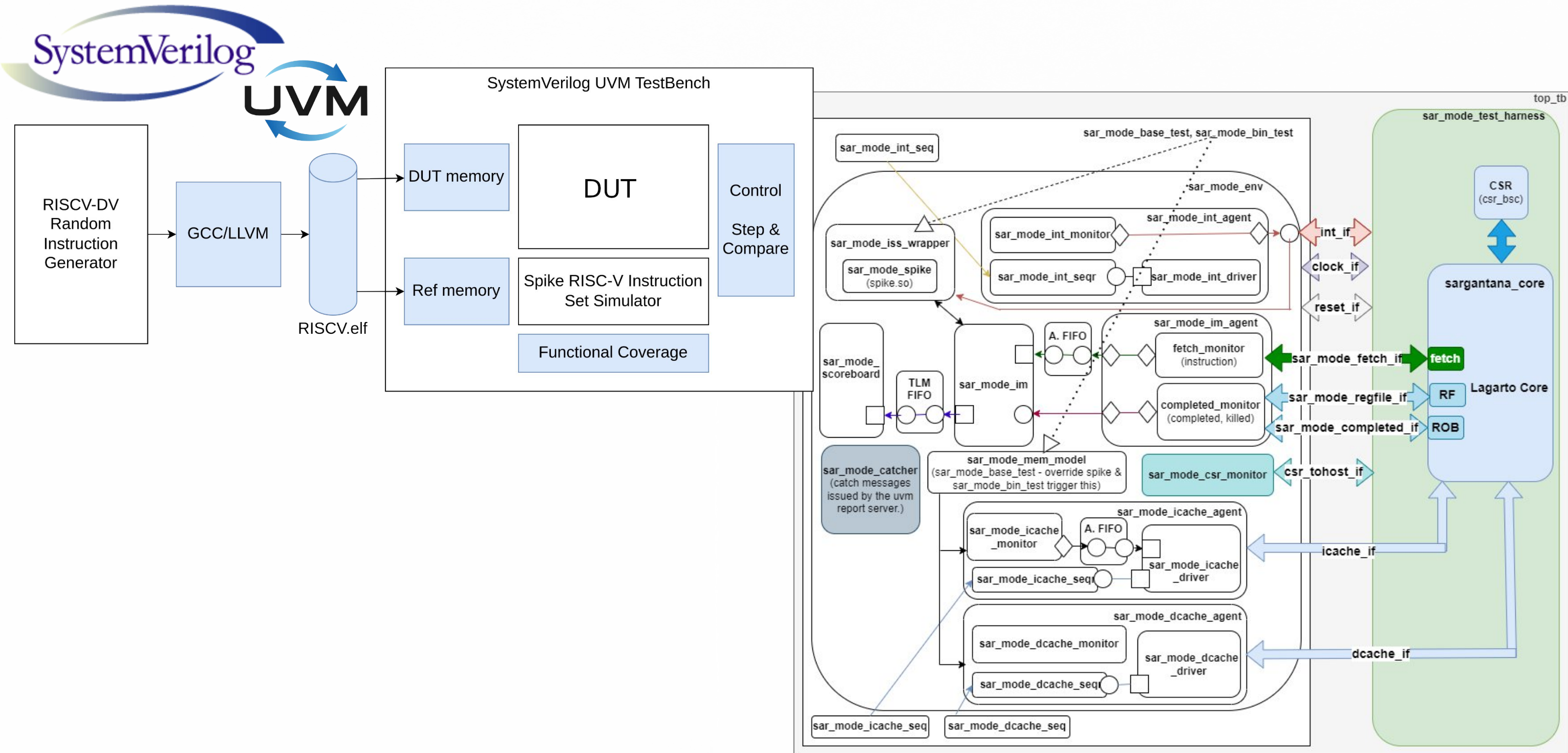
Summary: The RISC-V DV Mindset

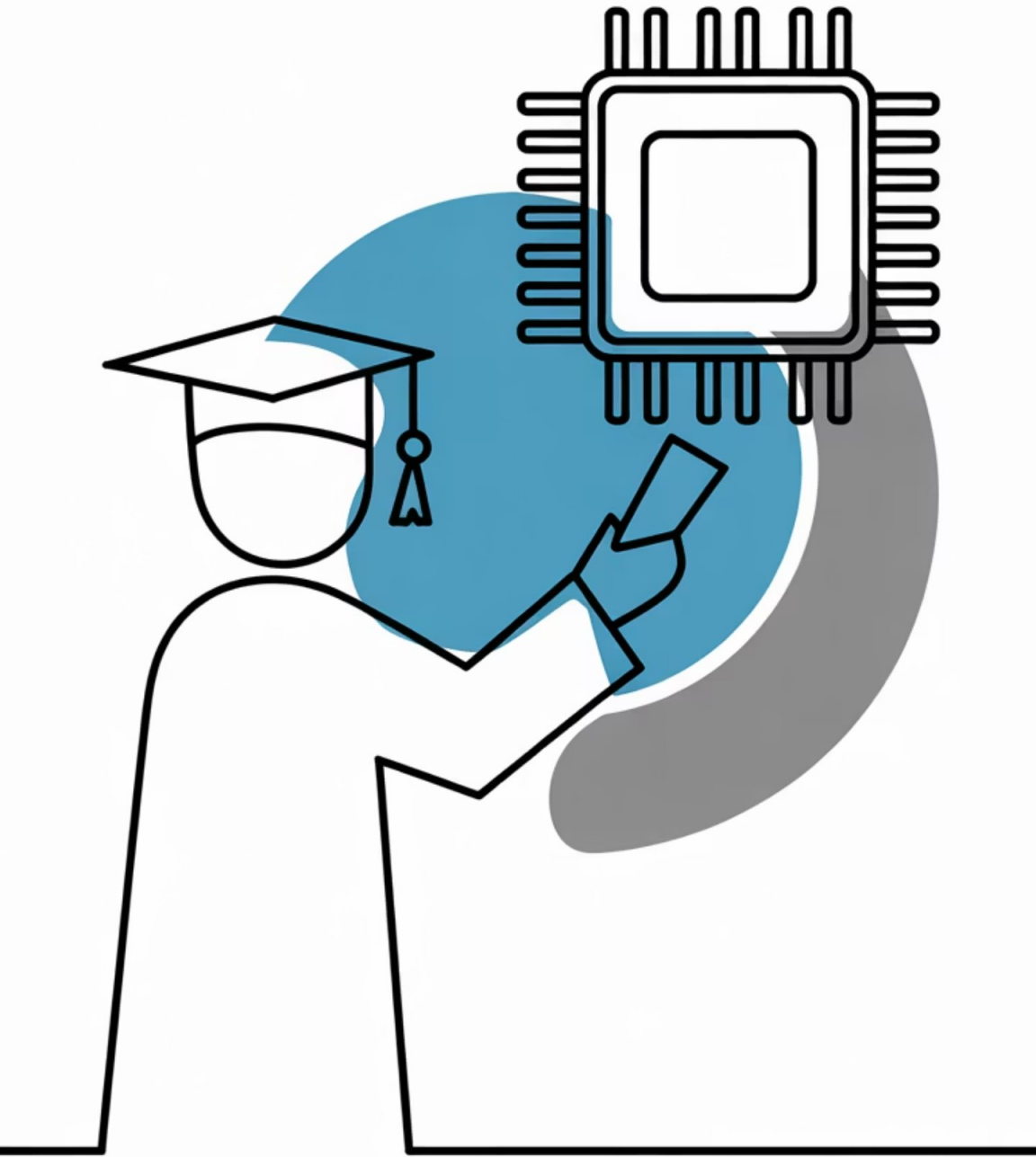
Successfully verifying a RISC-V core requires a blend of deep architectural knowledge and rigorous software engineering.



By mastering these modules, you are equipped to tackle the verification challenges of the next generation of open-standard processors.

Example: UVM for a core





Conclusion

This talk has provided a light overview of the main aspects and challenges of functional verification with RISC-V designs. Main takeaways:

- It is important
- It is hard
- It is fun (for some of us)

Next Steps

- Practice with Spike and RISC-V DV frameworks
- Build UVM testbenches for RISC-V IP
- Study open-source cores (Sargantana, CV32E40P, Rocket)
- Contribute to riscv-arch-test compliance suites

Key Resources

- RISC-V Specifications (Volume I & II)
- riscv.org for latest extensions and profiles
- OpenHW Group and OpenTitan verification repositories
- RISC-V International working groups
- Soon: Core-UVM and CPU-SS-UVM for Sargantana

Welcome to the RISC-V verification community. Your expertise will help ensure the correctness of the open-source hardware revolution.